

RAPPORT DE STAGE

Conception et développement d'une application web de gestion du planning

Home Office et de suivi du travail à distance

**Titre officiel : [INFORMATION À COMPLÉTER : titre officiel validé par
l'établissement et l'organisme d'accueil]**

Réalisé par : [INFORMATION À COMPLÉTER : nom et prénom de l'étudiant]

Matricule : [INFORMATION À COMPLÉTER : matricule de l'étudiant]

Établissement : [INFORMATION À COMPLÉTER : nom de l'établissement]

Filière et niveau : [INFORMATION À COMPLÉTER : filière et niveau d'études]

**Organisme d'accueil : LEONI — [INFORMATION À COMPLÉTER : entité et site
exacts]**

Département d'accueil : [INFORMATION À COMPLÉTER : département ou service]

Encadrant professionnel : [INFORMATION À COMPLÉTER : nom, fonction et qualité]

Encadrant académique : [INFORMATION À COMPLÉTER : nom, fonction et qualité]

**Période du stage : [INFORMATION À COMPLÉTER : date de début] —
[INFORMATION À COMPLÉTER : date de fin]**

Année universitaire : [INFORMATION À COMPLÉTER : année universitaire]

**[INFORMATION À COMPLÉTER : insérer les logos autorisés de l'établissement et de
l'organisme d'accueil]**

Dédicace

[INFORMATION À COMPLÉTER : texte personnel de dédicace, si cette page est exigée par le modèle académique]

Remerciements

Je tiens à remercier **[INFORMATION À COMPLÉTER : nom et fonction de l'encadrant professionnel]** pour **[INFORMATION À COMPLÉTER : accompagnement réellement reçu]**. J'adresse également mes remerciements à **[INFORMATION À COMPLÉTER : équipe, département ou personnes à citer avec leur autorisation]** ainsi qu'à **[INFORMATION À COMPLÉTER : encadrant académique]** pour **[INFORMATION À COMPLÉTER : nature de l'accompagnement académique]**.

Ce texte doit être personnalisé avant dépôt afin de refléter fidèlement les contributions et les conditions réelles du stage.

Résumé

Le présent rapport décrit une application web interne de gestion du planning Home Office et de suivi du travail à distance, identifiée techniquement sous le nom `leoni-planing`. La solution distingue deux rôles, **Team Leader** et **Data Cleansing**, puis centralise l'authentification, l'administration des utilisateurs, la sélection mensuelle d'un groupe, la génération des jours de travail distant, la consultation du planning, les demandes de congé, le suivi des sessions distantes, les exports et l'audit.

Le backend repose sur Node.js, Express et MySQL. Il combine des routes, des contrôleurs, des services et des modèles, avec des transactions sur les traitements sensibles. Le frontend est constitué de pages HTML, de styles CSS et de scripts JavaScript natifs s'appuyant sur Bootstrap. Les mécanismes observables de sécurité comprennent notamment le hashage bcrypt, les sessions stockées dans MySQL, la protection CSRF, Helmet, une politique de sécurité du contenu, la limitation de requêtes, la validation des entrées et une matrice de permissions.

La génération du planning applique une règle temporelle contrôlée par le serveur : du 25 au dernier jour du mois courant, seul le mois suivant peut être préparé. Une sélection mensuelle du groupe A ou B est exigée. Le groupe A reçoit les mercredis, les jeudis et les vendredis impairs du mois ; le groupe B reçoit les lundis, les mardis et les vendredis pairs. Les tests automatisés existants, exécutés le 15 juillet 2026 avec la commande `node --test tests/*.test.js`, ont produit 19 succès sur 19 tests. Leur couverture reste concentrée sur la fenêtre temporelle, la sélection mensuelle et la génération.

Le rapport distingue systématiquement les comportements confirmés par le code, les tests réellement exécutés et les validations fonctionnelles restant à effectuer. Les informations administratives, le contexte détaillé de l'organisme et l'état de déploiement ne pouvant pas être déduits du dépôt sont conservés sous forme de champs à compléter.

Mots-clés : application web, Home Office, planning, Node.js, Express, MySQL, sessions, contrôle d'accès, audit, travail à distance.

Abstract

This report presents an internal web application for Home Office scheduling and remote-work activity tracking, technically identified as `leoni-planing`. The solution defines two roles, **Team Leader** and **Data Cleansing**, and centralizes authentication, user administration, monthly group selection, remote-day generation, schedule consultation, leave requests, remote work sessions, exports, and audit events.

The backend relies on Node.js, Express, and MySQL. It combines routes, controllers, services, and models, with transactions for consistency-sensitive operations. The frontend consists of HTML pages, CSS styles, and native browser-side JavaScript using Bootstrap. Observable security controls include bcrypt password hashing, MySQL-backed sessions, CSRF protection, Helmet, a Content Security Policy, rate limiting, input validation, and permission-based access control.

Schedule generation is governed by a server-side time window: from the 25th through the actual last day of the current month, only the immediately following month can be prepared. A monthly selection of group A or B is mandatory. Group A receives Wednesdays, Thursdays, and odd-ranked Fridays; group B receives Mondays, Tuesdays, and even-ranked Fridays. The existing automated suite was run on 15 July 2026 with `node --test tests/*.test.js`; all 19 tests passed. Its scope is focused on the time window, monthly selection, and generation guards.

The report separates code-confirmed behavior, actually executed tests, and functional checks that remain to be performed. Administrative information, detailed organizational context, and deployment status that cannot be established from the repository are explicitly left to be completed.

Keywords: web application, Home Office, scheduling, Node.js, Express, MySQL, sessions, access control, audit, remote work.

Table des matières

Table des matières actualisable : clic droit > Mettre à jour le champ.

La version DOCX contient un champ de table des matières actualisable dans le traitement de texte. La version HTML propose une navigation vers les titres. La structure du document est la suivante :

1. Introduction générale
2. Chapitre 1 — Présentation de l'organisme et cadre du stage
3. Chapitre 2 — Analyse du système réalisé et conception
4. Chapitre 3 — Réalisation de l'application
5. Chapitre 4 — Tests et validation
6. Conclusion générale
7. Perspectives
8. Annexes
9. Références

Liste des figures

No	Figure prévue	Source ou statut
1	Cas d'utilisation global	diagrams/ 01_cas_utilisation_global.puml
2	Architecture globale	diagrams/ 02_architecture_globale.puml
3	Composants de l'application	diagrams/ 03_composants_application.puml
4	Modèle de données	diagrams/ 04_modele_donnees.puml
5	Séquence d'authentification	diagrams/ 05_sequence_authentification.puml
6	Séquence de changement du mot de passe	diagrams/ 06_sequence_changement_mot_de_passe.puml
7	Séquence de gestion d'un utilisateur	diagrams/ 07_sequence_gestion_utilisateur.puml
8	Séquence de sélection mensuelle	diagrams/ 08_sequence_selection_groupe.puml
9	Séquence de génération du planning	diagrams/ 09_sequence_generation_planning.puml
10	Séquence de consultation du planning	diagrams/ 10_sequence_consultation_planning.puml
11	Séquence de création d'une demande de congé	diagrams/ 11_sequence_demande_conge.puml
12	Séquence de traitement d'un congé	diagrams/ 12_sequence_traitement_conge.puml
13	Séquence d'une session de travail distant	diagrams/ 13_sequence_session_travail.puml
14	Séquence d'export	diagrams/ 14_sequence_export.puml
15	Séquence de consultation de l'audit	diagrams/ 15_sequence_audit.puml
16 à 31	Captures des interfaces existantes	À réaliser selon 03_plan_captures.md

Liste des tableaux

1. Acteurs et responsabilités observables.
2. Besoins fonctionnels déduits de la solution.
3. Besoins non fonctionnels observables.
4. Catalogue des règles de gestion.
5. Matrice synthétique des permissions.
6. Stack technique et versions.
7. Organisation des répertoires.
8. Tables et relations principales.
9. Interfaces et captures à insérer.
10. Résultats des tests automatisés.
11. Catalogue des validations manuelles.
12. Limites de couverture et perspectives.
13. Catalogue des routes.

Liste des abréviations

Abréviation	Signification
API	Application Programming Interface
CSP	Content Security Policy
CSRF	Cross-Site Request Forgery
CSV	Comma-Separated Values
DB / BDD	Database / Base de données
HO	Home Office
HTTP	Hypertext Transfer Protocol
MVC	Model–View–Controller
SQL	Structured Query Language
TL	Team Leader
UI	User Interface
UTC	Coordinated Universal Time
XLSX	Format de classeur Microsoft Excel

Introduction générale

Les organisations qui alternent présence sur site et travail à distance ont besoin de rendre lisibles les jours planifiés, les responsabilités de validation et les données utiles au suivi. Le code étudié matérialise ce besoin au moyen d'une application web qui regroupe les utilisateurs, les choix mensuels de groupe, les dates Home Office, les absences, les temps de travail réalisés, les exports et les événements d'audit. Le contexte détaillé qui a motivé cette réalisation n'est pas enregistré dans les sources : **[INFORMATION À COMPLÉTER :**

formulation du contexte métier validée par l'encadrant, sans donnée confidentielle].

La problématique technique traitée peut néanmoins être établie à partir du comportement de l'application : comment fournir, dans une même interface, une préparation mensuelle contrôlée du planning Home Office, une consultation adaptée aux droits de chaque rôle, un suivi non intrusif du temps actif et une traçabilité des opérations principales ? Cette question implique de coordonner plusieurs dimensions : la date métier, les permissions, les contraintes de base de données, les transactions, les validations serveur et l'expérience du navigateur.

La solution développée s'appuie sur un backend Node.js/Express connecté à MySQL et sur un frontend HTML/CSS/JavaScript. Deux rôles sont définis. Le Team Leader administre les comptes, consulte les données globales, traite les demandes, exporte et lit le journal. Le profil Data Cleansing accède principalement à ses propres données et opérations. Cette séparation s'appuie sur des permissions atomiques vérifiées côté serveur ; le menu est également adapté côté interface.

Le projet comprend une règle métier centrale de préparation du mois suivant. La fenêtre est calculée depuis l'horloge MySQL convertie dans le fuseau `Africa/Tunis`. Elle est ouverte à partir du 25 et se termine au dernier jour réel du mois. Pendant cette période, l'utilisateur choisit un groupe mensuel, puis le service génère les jours distants correspondants. Les contraintes uniques, les transactions et certains verrous limitent les incohérences concurrentes et les doublons.

Ce rapport adopte une démarche de rétrodocumentation : chaque description technique provient du code, du schéma, des migrations ou des tests actuels. L'inspection statique est distinguée de l'exécution. Ainsi, le résultat « 19 tests réussis » ne concerne que les trois fichiers de test effectivement lancés ; les autres parcours sont accompagnés de scénarios manuels « À valider ». Les informations administratives ou organisationnelles absentes sont signalées avec des placeholders.

Le premier chapitre présente le cadre du stage en séparant clairement les informations à compléter des éléments techniques confirmés. Le deuxième analyse les acteurs, les besoins déduits, les règles, l'architecture et les données. Le troisième expose la réalisation module par module. Le quatrième documente les validations exécutées et celles qui restent à conduire. La conclusion et les perspectives synthétisent les acquis, les limites observables et les consolidations possibles.

Chapitre 1 — Présentation de l'organisme et cadre du stage

1.1 Introduction

Ce chapitre situe le projet dans son cadre académique et professionnel. Les fichiers applicatifs ne constituent pas une source suffisante pour décrire l'histoire, l'organisation, les chiffres, les sites ou la stratégie de LEONI. Ces éléments sont donc laissés à compléter à partir de sources institutionnelles autorisées et d'informations validées par l'encadrement. Les seules affirmations fermes concernent l'application effectivement analysée.

1.2 Présentation de LEONI

LEONI est l'organisme d'accueil indiqué pour ce stage. Toute présentation institutionnelle détaillée doit être complétée et vérifiée avant remise :

- **[INFORMATION À COMPLÉTER : dénomination juridique exacte de l'entité d'accueil] ;**
- **[INFORMATION À COMPLÉTER : activité de l'entité et position dans l'organisation, validées par une source autorisée] ;**
- **[INFORMATION À COMPLÉTER : adresse et site du stage] ;**
- **[INFORMATION À COMPLÉTER : chiffres ou dates dont la publication est autorisée] ;**
- **[INFORMATION À COMPLÉTER : logo officiel et règles d'utilisation].**

Le texte final ne devra inclure aucune information interne, statistique ou affirmation institutionnelle sans validation.

1.3 Structure ou département d'accueil

[INFORMATION À COMPLÉTER : nom exact du département, mission, rattachement hiérarchique et composition utile à la compréhension du stage]

[INFORMATION À COMPLÉTER : relation vérifiée entre le département d'accueil et les deux catégories d'utilisateurs représentées dans l'application]

Le code emploie les libellés de rôle « Team Leader » et « Data Cleansing ». Cette présence prouve les profils fonctionnels de l'application, mais ne permet pas à elle seule de décrire l'organigramme réel ni le nombre de personnes concernées.

1.4 Cadre du stage

Le cadre administratif doit être renseigné avec les éléments suivants :

Élément	Valeur à valider
Nature du stage	[INFORMATION À COMPLÉTER : stage d'été, stage obligatoire ou autre désignation officielle]
Période	[INFORMATION À COMPLÉTER : dates exactes]
Durée	[INFORMATION À COMPLÉTER : durée officielle]
Étudiant	[INFORMATION À COMPLÉTER : identité et cursus]
Encadrement professionnel	[INFORMATION À COMPLÉTER : identité, fonction et rôle réel]
Encadrement académique	[INFORMATION À COMPLÉTER : identité et fonction]
Lieu et modalités	[INFORMATION À COMPLÉTER : site, présence, accès et contraintes pertinentes]
Confidentialité	[INFORMATION À COMPLÉTER : règles applicables au rapport et aux captures]

1.5 Contexte du projet

L'application actuelle regroupe plusieurs fonctions liées au planning Home Office. Elle conserve les comptes utilisateurs, associe une sélection de groupe à un utilisateur et à un mois, génère des dates distantes, affiche ces données sous forme tabulaire et calendrier, gère des congés, mesure des secondes actives pour les jours distants et fournit des exports. Elle offre également des indicateurs et un journal d'audit.

Ces fonctionnalités indiquent un besoin de centralisation et de contrôle cohérent des opérations. En revanche, le dépôt ne permet pas d'affirmer quel processus était employé auparavant, quel volume est traité ni si l'application est aujourd'hui déployée. Ces points doivent être validés : [INFORMATION À COMPLÉTER : contexte opérationnel et état réel d'utilisation].

1.6 Problématique traitée

La problématique déduite de la solution réalisée est la suivante : **comment centraliser la préparation et la consultation du planning Home Office, tout en contrôlant les droits, la période de génération, les opérations associées et leur traçabilité ?**

Cette problématique se décompose en plusieurs enjeux vérifiables : identifier les utilisateurs ; distinguer l'administration de l'usage personnel ; appliquer une date métier commune ; empêcher plusieurs générations pour le même mois ; fiabiliser les écritures liées ; préserver la

propriété des données individuelles ; calculer les heures réalisées ; permettre l'extraction des informations ; et conserver une trace des actions importantes.

1.7 Objectifs du projet réalisé

Les objectifs techniques observables, formulés depuis les fonctions présentes, sont :

1. authentifier les utilisateurs et imposer le renouvellement initial du mot de passe lorsque les indicateurs correspondants sont actifs ;
2. administrer les comptes et leur rôle sans supprimer physiquement les utilisateurs ;
3. autoriser une sélection mensuelle de groupe A ou B pendant une fenêtre serveur déterminée ;
4. générer automatiquement les jours Home Office du mois suivant selon le groupe sélectionné ;
5. consulter le planning sous plusieurs représentations et selon le périmètre autorisé ;
6. gérer le cycle des demandes de congé ;
7. suivre les sessions actives d'un utilisateur sur une date distante qui lui appartient ;
8. exporter les données de planning en CSV ou XLSX pour un rôle autorisé ;
9. journaliser les opérations majeures ;
10. appliquer des validations, permissions et traitements d'erreur partagés.

1.8 Organisation du travail

[INFORMATION À COMPLÉTER : méthode de travail réellement suivie, outils de suivi, fréquence des échanges et méthode de validation]

Il ne serait pas exact de qualifier la démarche d'Agile, Scrum, cycle en V ou autre sans preuve externe au code. La documentation technique montre uniquement une organisation du logiciel par répertoires, des migrations successives numérotées et trois fichiers de tests automatisés. Ces éléments ne démontrent pas, à eux seuls, une méthodologie de projet.

1.9 Déroulement du stage

[INFORMATION À COMPLÉTER : déroulement chronologique réel du stage, jalons, tâches, revues et livraisons]

Le déroulement final peut être présenté sous forme de tableau après validation :

Période	Activité réelle	Livrable ou résultat	Validation obtenue
[INFORMATION À COMPLÉTER : période 1]	[INFORMATION À COMPLÉTER : activité]	[INFORMATION À COMPLÉTER : résultat]	[INFORMATION À COMPLÉTER : validation]

Période	Activité réelle	Livrable ou résultat	Validation obtenue
[INFORMATION À COMPLÉTER : période 2]	[INFORMATION À COMPLÉTER : activité]	[INFORMATION À COMPLÉTER : résultat]	[INFORMATION À COMPLÉTER : validation]
[INFORMATION À COMPLÉTER : période 3]	[INFORMATION À COMPLÉTER : activité]	[INFORMATION À COMPLÉTER : résultat]	[INFORMATION À COMPLÉTER : validation]

1.10 Conclusion

Le cadre administratif et institutionnel nécessite une contribution directe de l'étudiant et de ses encadrants. Le périmètre technique, lui, est clairement identifiable : une application web de planification Home Office et de suivi de fonctions connexes. Le chapitre suivant transforme ce périmètre implémenté en acteurs, besoins fonctionnels, règles de gestion, architecture et modèle de données, sans extrapoler au-delà des sources.

Chapitre 2 — Analyse du système réalisé et conception

2.1 Introduction

L'analyse présentée ici part du produit réalisé. Elle reformule en besoins et en règles les comportements confirmés par les routes, les contrôleurs, les services, les modèles, le schéma SQL, les vues et les tests. Cette approche permet de documenter précisément la solution sans attribuer au projet un objectif ou un processus qui ne serait pas observable.

2.2 Présentation fonctionnelle de la solution

La solution est une application web à pages statiques servies par Express et alimentées par des API JSON. Un utilisateur se connecte par e-mail et mot de passe. Selon son rôle et l'état de son onboarding, il accède à un tableau de bord et aux modules autorisés. Le Team Leader possède un périmètre d'administration et de consultation globale ; le profil Data Cleansing dispose d'un périmètre centré sur ses opérations.

Le planning est préparé par mois. Une date serveur de référence détermine si la période de préparation est ouverte. L'utilisateur sélectionne alors un groupe pour le mois suivant. Ce groupe fixe les jours de la semaine et les rangs de vendredi à générer. Une fois le planning créé, la sélection est verrouillée pour préserver la cohérence entre le choix et les lignes produites.

Les modules complémentaires restent reliés au même compte : congés personnels, sessions de travail à distance, heures réelles, export des données et journalisation. Ils forment un ensemble cohérent autour des personnes, des dates et de la traçabilité, même si toutes les couches techniques ne sont pas employées de manière uniforme.

2.3 Identification des acteurs

Acteur	Responsabilités et accès observables	Restrictions principales
Visiteur non authentifié	Afficher la page de connexion, soumettre un e-mail et un mot de passe, demander l'état de session	Aucun accès aux pages métiers ; limitation spécifique des tentatives de connexion
Utilisateur en onboarding	Consulter la page de changement du mot de passe et remplacer son mot de passe temporaire	Redirigé hors des autres pages tant que <code>first_login</code> ou <code>must_change_password</code> reste actif
Data Cleansing	Consulter son planning et le calendrier, sélectionner son groupe mensuel, générer son planning, gérer ses demandes de congé, utiliser ses sessions de travail, changer son mot de passe	Pas d'administration des comptes, pas d'export, pas de lecture des journaux, pas de traitement global des congés
Team Leader	Réaliser les opérations personnelles et	Ne peut pas approuver ou rejeter sa

Acteur	Responsabilités et accès observables	Restrictions principales
	administrer les utilisateurs ; lire/générer les plannings autorisés ; lire les statuts mensuels ; voir les synthèses ; gérer les congés ; exporter ; lire l'audit	propre demande ; reste soumis aux validations et à la fenêtre de génération
Horloge et ordonnanceur internes	Calculer la date métier, expirer les sessions obsolètes au démarrage et périodiquement	Acteur technique sans interface ; dépend du serveur et de MySQL

Le rôle n'est pas seulement utilisé pour masquer des éléments du menu. Les routes d'administration, d'export, d'audit et de congé appliquent `requirePermission`. Les services de planning et de sessions complètent ces vérifications par des contrôles de propriété ou de périmètre.

2.4 Modules fonctionnels

Module	Finalité confirmée	Acteurs	Persistance principale
Authentification	Ouvrir et fermer une session après vérification du mot de passe	Tous les comptes actifs	<code>users</code> et magasin de sessions MySQL
Onboarding du mot de passe	Bloquer les pages métiers jusqu'au renouvellement requis	Utilisateur concerné	<code>users</code>
Utilisateurs	Lister, créer, mettre à jour et désactiver logiquement un compte	Team Leader	<code>users</code> , <code>audit_logs</code>
Contrôle d'accès	Associer des permissions atomiques aux deux rôles	Serveur et frontend	Constantes et session
Sélection mensuelle	Enregistrer le groupe A/B d'un utilisateur pour un mois cible	Tous pour soi ; lecture globale TL	<code>monthly_group_selections</code>
Génération	Calculer et insérer les jours distants autorisés	Tous pour soi ; TL selon permission	<code>planning</code> , <code>monthly_group_selections</code>
Consultation	Filtrer et afficher les lignes de planning	Tous selon périmètre	<code>planning</code> , <code>users</code>
Calendrier	Regrouper les jours par utilisateur et par mois	Tous selon périmètre	<code>planning</code> , <code>users</code>
Tableau de bord	Fournir des statistiques et des états mensuels	Utilisateurs authentifiés avec onboarding terminé	Agrégats sur <code>users</code> , <code>planning</code> , sélections
Congés	Créer, annuler, approuver ou rejeter selon le rôle et le statut	Tous pour soi ; TL pour traitement	<code>leave_requests</code> , <code>audit_logs</code>
Sessions de travail	Démarrer/repandre, maintenir, mettre en pause, terminer ou expirer une session distante	Propriétaire ; synthèse TL	<code>work_sessions</code> , <code>planning</code> , <code>audit_logs</code>
Export	Produire un CSV ou un classeur XLSX filtré	Team Leader	Lecture de <code>planning</code> et <code>users</code> , audit
Audit	Enregistrer et afficher les opérations principales	Écriture applicative ; lecture TL	<code>audit_logs</code>

2.5 Besoins fonctionnels déduits de l'application réalisée

Les priorités ci-dessous sont des priorités techniques déduites des protections et dépendances du code. Elles ne représentent pas une hiérarchisation contractuelle.

ID	Titre	Acteur	Description et préconditions	Résultat	Priorité technique	Preuve source
BF-01	Se connecter	Compte actif	E-mail valide et mot de passe soumis à l'API	Session contenant un profil réduit ou erreur d'authentification	Critique	authRoutes.js, authController.js
BF-02	Terminer l'onboarding	Utilisateur concerné	Session authentifiée, ancien mot de passe et nouveau mot de passe valide	Hash remplacé et indicateurs d'onboarding désactivés	Critique	auth.js, authValidation.js, authController.js
BF-03	Se déconnecter	Utilisateur authentifié	Session existante	Session détruite et cookie effacé	Élevée	authController.js
BF-04	Administrer un utilisateur	Team Leader	Permission d'action et données valides	Compte créé, modifié ou désactivé logiquement	Élevée	userRoutes.js, userController.js
BF-05	Consulter la fenêtre de génération	Utilisateur autorisé	Session et onboarding terminés	État de la fenêtre, date métier et mois cible renvoyés	Critique	planningRoutes.js, PlanningGenerationWindowService.js
BF-06	Choisir un groupe mensuel	Utilisateur pour lui-même	Fenêtre ouverte, mois suivant, groupe 1 ou 2, aucun planning verrouillant	Sélection créée ou mise à jour	Critique	monthlyGroupSelectionRoutes.js, MonthlyGroupSelectionService.js
BF-07	Lire les sélections du mois	Team Leader	Permission de lecture globale et mois valide	État des choix mensuels des comptes concernés	Moyenne	monthlyGroupSelectionController.js
BF-08	Générer son planning	Utilisateur autorisé	Fenêtre ouverte, mois suivant, sélection présente, planning absent	Lignes remote insérées pour les dates calculées	Critique	PlanningService.js
BF-09	Générer pour un autre compte	Team Leader	Permission globale et sélection du compte cible	Planning du compte cible créé	Élevée	planningController.js, PlanningService.js
BF-10	Consulter le planning	Utilisateur authentifié	Filtres valides ; périmètre appliqué par le service	Lignes autorisées renvoyées	Élevée	planningRoutes.js, PlanningService.js
BF-11	Consulter le calendrier	Utilisateur authentifié	Mois ou filtres autorisés	Jours regroupés pour l'affichage calendaire	Moyenne	planningController.js, calendar.js
BF-12	Lire le tableau de bord	Utilisateur authentifié	Onboarding terminé	Agrégats et données d'état affichés	Moyenne	dashboardRoutes.js, dashboardController.js
BF-13	Créer une demande de congé	Utilisateur	Dates, type et motif valides, sans chevauchement actif	Demande pending créée	Élevée	leaveRequestRoutes.js, LeaveRequestService.js
BF-14	Annuler sa demande	Propriétaire	Demande encore pending	Statut cancelled	Moyenne	LeaveRequestService.js
BF-15	Traiter une	Team Leader	Demande d'un	Statut approved	Élevée	LeaveRequestService.js

ID	Titre	Acteur	Description et préconditions	Résultat	Priorité technique	Preuve source
	demande		autre utilisateur encore pending	ou rejected , réviseur mémorisé		
BF-16	Démarrer une session distante	Utilisateur	Planning remote personnel pour la date serveur	Session créée ou reprise	Élevée	<code>WorkSessionService.js</code>
BF-17	Maintenir ou suspendre une session	Propriétaire	Session valide	Heartbeat, pause ou cumul de secondes	Élevée	<code>workSessionRoutes.js</code> , <code>WorkSessionService.js</code>
BF-18	Terminer ou expirer une session	Propriétaire ou tâche interne	Session existante ou heartbeat obsolète	Session clôturée et heures réelles recalculées	Élevée	<code>WorkSessionService.js</code> , <code>server.js</code>
BF-19	Exporter le planning	Team Leader	Permission CSV/XLSX et filtres reçus	Téléchargement du fichier et événement d'audit	Moyenne	<code>exportRoutes.js</code> , <code>exportController.js</code>
BF-20	Consulter l'audit	Team Leader	Permission <code>audit.read</code>	Jusqu'aux cent événements les plus récents	Moyenne	<code>logRoutes.js</code> , <code>logController.js</code>

2.6 Besoins non fonctionnels observables

ID	Besoin observable	Mise en œuvre vérifiée	Réserve
BNF-01	Confidentialité des mots de passe	Hashage bcrypt avec dix tours ; mots de passe non renvoyés dans la session	Aucun audit cryptographique exécuté
BNF-02	Contrôle d'accès	Authentification, onboarding, permissions et contrôles de propriété	Quelques routes de lecture s'appuient davantage sur le service que sur une permission déclarée dans la route
BNF-03	Intégrité des données	Contraintes uniques, clés étrangères, transactions et verrous sur des flux sensibles	Validation avec une instance MySQL non exécutée dans cette mission
BNF-04	Protection HTTP	Helmet, CSP, CSRF, cookies configurés, limitation de requêtes	Les résultats d'un test dynamique des en-têtes ne sont pas disponibles
BNF-05	Traçabilité	Table d'audit et actions nommées pour les opérations principales	La complétude de l'audit n'est pas garantie pour chaque lecture ou erreur
BNF-06	Maintenabilité	Découpage routes/contrôleurs/services/modèles et constantes partagées	Architecture hétérogène : certains contrôleurs requêtent directement MySQL
BNF-07	Compatibilité responsive	Feuille CSS avec points de rupture, composants Bootstrap et styles adaptatifs	Aucun test multi-navigateur automatisé identifié
BNF-08	Résilience des sessions de travail	Heartbeat, expiration, nettoyage au démarrage et périodique	Dépend de l'exécution continue du processus serveur
BNF-09	Testabilité	Services de fenêtre isolés et tests natifs Node avec substituts	Couverture limitée à trois fichiers et script <code>npm test</code> non relié à la suite
BNF-10	Portabilité de configuration	Variables d'environnement référencées et exemple de noms génériques	Version Node exigée et procédure de déploiement non précisées dans le projet

2.7 Règles de gestion implémentées

ID	Règle confirmée par le code
RG-01	L'identifiant de connexion soumis par l'interface et validé par le serveur est une adresse e-mail.
RG-02	Seul un utilisateur non supprimé logiquement peut ouvrir une session.
RG-03	Le mot de passe est vérifié avec bcrypt et les nouveaux mots de passe doivent contenir au moins huit caractères.
RG-04	Tant que <code>first_login</code> ou <code>must_change_password</code> est actif, l'utilisateur est limité au changement du mot de passe.
RG-05	Le changement personnel exige l'ancien mot de passe correct.
RG-06	Les rôles reconnus sont exactement <code>Team Leader</code> et <code>Data Cleansing</code> .
RG-07	L'administration des utilisateurs est accordée au Team Leader par permissions atomiques.
RG-08	La suppression d'un utilisateur est logique : <code>is_deleted</code> et <code>deleted_at</code> sont mis à jour.
RG-09	Un nom d'utilisateur, un e-mail et un matricule doivent rester uniques.
RG-10	La date métier de préparation est déterminée depuis MySQL puis convertie dans <code>Africa/Tunis</code> .
RG-11	La fenêtre mensuelle est fermée du 1er au 24 et ouverte du 25 au dernier jour réel du mois.
RG-12	Seul le mois immédiatement suivant la date métier peut être sélectionné ou généré.
RG-13	Un groupe mensuel vaut 1 (A) ou 2 (B), avec une seule sélection par utilisateur et mois.
RG-14	Un utilisateur ne peut modifier que sa propre sélection mensuelle.
RG-15	La sélection est verrouillée dès qu'un planning existe pour l'utilisateur et le mois.
RG-16	La génération est refusée lorsqu'aucune sélection mensuelle n'est disponible.
RG-17	La génération est refusée lorsqu'un planning du même utilisateur et du même mois existe déjà.
RG-18	Le groupe A reçoit les mercredis, les jeudis et les premier, troisième et éventuel cinquième vendredis.
RG-19	Le groupe B reçoit les lundis, les mardis et les deuxième et quatrième vendredis.
RG-20	Le générateur insère les dates calculées avec le statut <code>remote</code> , zéro heure réalisée et huit heures prévues.
RG-21	Une seule ligne de planning peut exister pour un utilisateur et une date.
RG-22	Un utilisateur sans droit de lecture globale ne reçoit que son propre périmètre dans les services de planning.
RG-23	Une demande de congé doit avoir une plage valide, un type autorisé et un texte respectant les limites de validation.
RG-24	Deux demandes du même utilisateur ne peuvent se chevaucher si l'une est <code>pending</code> ou <code>approved</code> .

ID	Règle confirmée par le code
RG-25	Le propriétaire peut annuler uniquement sa demande encore <code>pending</code> .
RG-26	Le Team Leader ne peut traiter que les demandes <code>pending</code> d'un autre utilisateur.
RG-27	Une session de travail démarre ou reprend seulement pour un planning personnel <code>remote</code> à la date serveur.
RG-28	La contrainte <code>active_slot</code> vise à maintenir une seule session active pour l'utilisateur, le planning et la date.
RG-29	Le navigateur envoie un heartbeat toutes les 60 secondes pendant l'activité.
RG-30	Une inactivité détectée côté navigateur après cinq minutes provoque une demande de pause.
RG-31	Le serveur accorde une grâce de 120 secondes autour du heartbeat et recherche les sessions obsolètes toutes les deux minutes.
RG-32	Les sessions possèdent les statuts <code>active</code> , <code>paused</code> , <code>ended</code> et <code>expired</code> .
RG-33	Les secondes actives sont cumulées lors des transitions et converties en heures réelles.
RG-34	Les heures réelles d'une journée sont plafonnées à huit heures ; les heures prévues restent séparées.
RG-35	Les exports CSV et XLSX sont réservés au rôle possédant les permissions correspondantes.
RG-36	Le journal présenté à l'interface est limité aux cent événements les plus récents.

2.8 Cas d'utilisation global

Le cas d'utilisation global associe les fonctions personnelles aux deux rôles et les fonctions d'administration au Team Leader. Le fichier PlantUML correspondant représente les dépendances sans introduire d'acteur externe non observé.

[CAPTURE À INSÉRER : rendu du diagramme `01\_cas\_utilisation\_global.puml` — Cas d'utilisation global]

Figure 1 — Cas d'utilisation global de l'application.

2.9 Description détaillée des cas d'utilisation

2.9.1 CU-01 — S'authentifier

- **Acteur** : utilisateur possédant un compte actif.
- **Objectif** : ouvrir une session applicative.
- **Préconditions** : e-mail au format valide, compte non supprimé, mot de passe connu.

- **Scénario nominal** : l'utilisateur soumet le formulaire ; l'API cherche le compte par e-mail ; bcrypt compare le secret ; le serveur régénère ou renseigne la session avec un profil réduit ; l'interface redirige vers le changement du mot de passe ou le tableau de bord.
- **Alternatives** : session déjà active ; indicateurs d'onboarding actifs.
- **Exceptions** : e-mail mal formé, compte absent, mot de passe incorrect, limitation de requêtes, erreur interne.
- **Postcondition** : session authentifiée ou absence de session.

2.9.2 CU-02 — Sélectionner le groupe mensuel

- **Acteur** : Team Leader ou Data Cleansing pour son propre compte.
- **Objectif** : enregistrer le groupe A ou B du prochain mois.
- **Préconditions** : session valide, onboarding terminé, fenêtre ouverte, mois cible exact, planning absent.
- **Scénario nominal** : l'interface lit la fenêtre ; l'utilisateur choisit A ou B ; le contrôleur transmet son identifiant au service ; le service verrouille les données nécessaires, vérifie la date et le planning, puis insère ou met à jour la sélection dans une transaction ; une action d'audit est créée.
- **Alternatives** : modification d'une sélection existante avant génération.
- **Exceptions** : fenêtre fermée, mois invalide, groupe invalide ou sélection verrouillée.
- **Postcondition** : une seule sélection existe pour l'utilisateur et le mois.

2.9.3 CU-03 — Générer le planning

- **Acteur** : utilisateur pour lui-même ; Team Leader pour une cible autorisée.
- **Objectif** : créer les lignes Home Office du prochain mois.
- **Préconditions** : fenêtre ouverte, mois cible, sélection présente et absence de planning mensuel.
- **Scénario nominal** : le service ouvre une transaction ; reverrouille et revérifie la fenêtre ; charge la sélection ; refuse un planning existant ; calcule les dates selon le groupe ; insère le lot de lignes remote ; écrit l'audit ; valide la transaction.
- **Alternatives** : génération pour un autre utilisateur par le Team Leader.
- **Exceptions** : sélection absente, cible non autorisée, doublon, erreur SQL ou fenêtre fermée.
- **Postcondition** : les jours distants calculés existent une seule fois.

2.9.4 CU-04 — Traiter une demande de congé

- **Acteur** : Team Leader.
- **Objectif** : approuver ou rejeter une demande en attente.
- **Préconditions** : permission de gestion, demande existante, demande d'un autre utilisateur, statut `pending`.
- **Scénario nominal** : le Team Leader choisit une décision et éventuellement un commentaire ; le service charge et verrouille la demande ; vérifie le propriétaire et le statut ; renseigne la décision, le réviseur et l'horodatage ; écrit l'audit ; valide la transaction.
- **Alternatives** : approbation ou rejet.
- **Exceptions** : auto-traitement, demande absente, demande déjà décidée ou données invalides.
- **Postcondition** : statut terminal `approved` ou `rejected`.

2.9.5 CU-05 — Suivre une session distante

- **Acteur** : utilisateur propriétaire du planning.
- **Objectif** : comptabiliser les périodes actives d'une journée distante.
- **Préconditions** : ligne `remote` pour la date serveur et le compte connecté.
- **Scénario nominal** : l'interface demande le démarrage automatique ; le service crée ou reprend la session ; le navigateur envoie des heartbeats ; une pause ou une fin cumule le temps ; le service met à jour `planning.work_hour` avec un plafond de huit heures.
- **Alternatives** : reprise d'une session en pause ; expiration automatique en cas de heartbeat obsolète.
- **Exceptions** : planning non éligible, session d'un autre utilisateur, session absente ou conflit d'unicité.
- **Postcondition** : statut de session et heures réelles cohérents avec les périodes enregistrées.

2.10 Architecture globale

L'architecture est monolithique et organisée par responsabilités. Le navigateur reçoit des pages et des assets depuis Express, puis appelle les endpoints `/api`. Le serveur enchaîne des middlewares transversaux, des routes, des contrôleurs et, pour plusieurs domaines, des services et modèles. Le pool `mysql2/promise` fournit l'accès à MySQL. Un magasin séparé, géré par `express-mysql-session`, persiste les sessions HTTP.

Le schéma `route` → `contrôleur` → `service` → `modèle` → `MySQL` décrit correctement la génération, la sélection mensuelle, les congés et les sessions de travail. Il ne doit toutefois pas

être présenté comme universel : l'authentification, les utilisateurs, le tableau de bord, l'export et les journaux contiennent des requêtes SQL directement dans les contrôleurs ou des utilitaires. L'architecture est donc « en couches inspirée de MVC, avec accès aux données hétérogène ».

[CAPTURE À INSÉRER : rendu du diagramme `02\_architecture\_globale.puml` — Architecture globale]

Figure 2 — Flux global du navigateur vers MySQL.

2.11 Architecture backend

`server.js` charge la configuration, instancie Express, active la confiance dans un proxy, applique Helmet et la CSP, Morgan, les limiteurs, les analyseurs de cookies et de corps, les fichiers statiques, la session MySQL, puis CSRF. Les routes de pages précèdent les groupes d'API. Une route 404 et un gestionnaire global terminent la chaîne. Après l'écoute, le nettoyage des sessions de travail obsolètes est lancé au démarrage puis à intervalle régulier.

Les validations `express-validator` sont placées dans `validations/` et consolidées par `middlewares/validate.js`. Les erreurs attendues peuvent être portées par les classes de `utils/errors.js`; `utils/asyncHandler.js` facilite leur propagation. Plusieurs services utilisent une connexion dédiée au pool pour commencer, valider ou annuler une transaction, ce qui évite de répartir une opération atomique sur plusieurs connexions.

[CAPTURE À INSÉRER : rendu du diagramme `03\_composants\_application.puml` — Composants backend et frontend]

Figure 3 — Principaux composants du monolithe.

2.12 Organisation frontend

Chaque page métier est un fichier HTML distinct. `layout.js` construit les éléments partagés, charge l'état de session et filtre la navigation à partir des permissions. `api.js` fournit un client commun pour les appels `fetch`, obtient le jeton CSRF à partir de l'API de session et normalise les erreurs. Les scripts propres aux pages assurent les formulaires, les filtres, les tableaux, les modales, les messages et les appels aux endpoints.

La feuille `leoni.css` définit l'identité visuelle, les états, la grille, les tableaux et des adaptations aux largeurs 1199, 991 et 575 pixels. Elle contient aussi des styles `focus-visible` et respecte `prefers-reduced-motion`. Bootstrap 5.3.3, Font Awesome 6.5.2 et la police Inter sont chargés depuis des ressources CDN par les pages. Cette dépendance réseau doit être prise en compte pour un usage isolé.

2.13 Modèle de données

Le schéma métier de premier niveau comprend six tables. Le magasin de sessions HTTP est créé et géré à l'exécution par la bibliothèque de session ; sa structure exacte ne fait pas partie du schéma métier de premier niveau décrit ici.

Table	Finalité	Clés et relations	Contraintes remarquables
users	Comptes, identité métier, rôle et onboarding	PK id	username, email, matricule uniques ; rôle énuméré ; suppression logique ; index groupe/rôle/suppression
monthly_group_selections	Groupe d'un utilisateur pour un mois	PK id, FK user_id → users avec cascade	month_key au format année-mois ; groupe 1/2 ; unicité utilisateur/mois ; index mois et groupe
planning	Journées planifiées et heures	PK id, FK user_id → users avec cascade	unicité utilisateur/date ; statut onsite/remote ; mois indexé ; heures prévues/réelles séparées
work_sessions	Segments de suivi des journées distantes	PK id, FK utilisateur en cascade, FK planning avec mise à null	statuts énumérés ; heartbeat ; secondes ; active_slot et unicité de la session active
leave_requests	Demandes et décisions de congé	PK id, FK demandeur en cascade, FK réviseur avec mise à null	type et statut énumérés ; dates ; index demandeur/statut/période
audit_logs	Trace horodatée des actions	PK id, FK utilisateur avec mise à null	action, détails, adresse IP, index date/action

Les migrations numérotées complètent le schéma avec les colonnes d'entreprise, les congés, le champ horaire, les sessions de travail, leur durcissement, la date de mise à jour du planning et les sélections mensuelles. Le code d'initialisation exécute aussi des opérations de création ou de migration au démarrage ; cette caractéristique est reprise comme limite opérationnelle dans le chapitre 3.

[CAPTURE À INSÉRER : rendu du diagramme `04\_modele\_donnees.puml` — Modèle relationnel]

Figure 4 — Relations des six tables métier.

2.14 Contrôle d'accès

Action ou module	Team Leader	Data Cleansing	Contrôle principal
Tableau de bord	Oui	Oui	Authentification et onboarding ; permission de menu
Administration des utilisateurs	Oui	Non	Permissions users.* sur pages et API
Lecture de son planning	Oui	Oui	Permission et périmètre de service
Lecture globale du planning	Oui	Non	Permission disponible et

Action ou module	Team Leader	Data Cleansing	Contrôle principal
			filtrage du service
Génération pour soi	Oui	Oui	<code>planning.generate.own</code>
Génération pour autrui	Oui	Non	<code>planning.generate.all</code> vérifiée dans le contrôleur/service
Sélection mensuelle personnelle	Oui	Oui	Permissions lecture/écriture propre
Lecture globale des sélections	Oui	Non	<code>monthly_group_selection.read.all</code>
Congés personnels	Oui	Oui	<code>leave_requests.read.own</code>
Gestion des congés	Oui	Non	<code>leave_requests.manage</code> et règle anti-auto-traitement
Synthèse des sessions	Oui	Non	<code>work_sessions.read.summary</code>
Export CSV/XLSX	Oui	Non	<code>export.csv</code> et <code>export.xlsx</code>
Lecture de l'audit	Oui	Non	<code>audit.read</code>
Changement personnel du mot de passe	Oui	Oui	Authentification uniquement

Le menu n'est qu'une aide d'interface. La protection effective repose sur les middlewares et les contrôles métier. Trois endpoints de lecture du planning (`/calendar`, `/`, `/all` et la route utilisateur selon leur ordre) n'emploient pas tous une permission explicite dans le fichier de route ; le périmètre est ensuite restreint par la logique du contrôleur/service. Cette dissociation fonctionne comme défense fonctionnelle, mais rend la politique moins immédiatement lisible.

2.15 Conception de la sécurité

La conception combine plusieurs couches. L'authentification recherche le compte par e-mail, exclut les comptes supprimés et compare un hash bcrypt. La session stocke une représentation limitée du compte et le cookie est configuré `HttpOnly`, `SameSite=Lax`, avec l'attribut `Secure` activé en environnement de production et une durée maximale de huit heures.

Les requêtes qui modifient l'état passent par la protection CSRF globale, après initialisation de la session. Helmet définit des en-têtes de sécurité et une CSP explicite qui autorise les ressources nécessaires aux pages. Un limiteur général couvre l'API et un limiteur plus strict cible la connexion. Les entrées sont validées côté serveur. Les requêtes SQL de premier niveau inspectées utilisent des paramètres pour les valeurs ; les clauses dynamiques observées sont assemblées à partir de fragments contrôlés par le code.

Les permissions atomiques évitent de disperser des comparaisons de libellés de rôle, tandis que les services vérifient la propriété des plannings, demandes et sessions. Les opérations principales alimentent `audit_logs`. Le gestionnaire global masque les détails internes des erreurs inattendues. Ces constats décrivent des mécanismes présents ; ils ne constituent ni un test d'intrusion ni une certification de sécurité.

Deux réserves ressortent directement du code. Les messages d'échec de connexion distinguent l'absence de compte d'un mot de passe erroné, ce qui peut faciliter l'énumération d'adresses. Le démarrage de la couche de données comporte un chemin d'initialisation et d'amorçage codé dans l'application ; aucun identifiant n'est reproduit ici, mais ce mécanisme gagnerait à être externalisé et contrôlé par une procédure de migration dédiée.

2.16 Conclusion

Le système réalisé couvre un cycle complet autour du planning Home Office : identité, groupe mensuel, génération, consultation, activités associées et traçabilité. Les rôles et les règles sont suffisamment explicites dans le code pour construire un modèle fonctionnel détaillé. L'architecture apporte des séparations utiles et des transactions sur les écritures sensibles, tout en conservant une hétérogénéité d'accès aux données. Le chapitre suivant examine concrètement la mise en œuvre de chaque composant et de chaque interface.

Chapitre 3 — Réalisation de l'application

3.1 Introduction

Ce chapitre décrit l'implémentation actuelle, depuis le démarrage du serveur jusqu'aux interfaces. Pour chaque module, il met en relation la page, l'API, la couche métier et les données. Il précise également les erreurs contrôlées, les événements d'audit et les limites que le code permet d'établir.

3.2 Environnement technique

Le projet applicatif se trouve dans `leoni-planing/` et déclare le nom `leoni-planing`, la version applicative 1.0.0 et le mode de modules CommonJS. La version minimale de Node.js n'est pas fixée par un champ `engines`. L'exécution documentaire des tests a utilisé Node.js v24.16.0 et npm 11.13.0 ; ces versions décrivent l'environnement du test, pas une contrainte officielle du projet.

Technologie	Version exacte détectée	Rôle réel	Preuve
Express	5.2.1	Serveur HTTP, pages, API et middlewares	verrou de dépendances
mysql2	3.22.5	Pool MySQL, promesses, transactions et requêtes préparées	verrou de dépendances, <code>config/db.js</code>
bcrypt	6.0.0	Hashage et comparaison des mots de passe	verrou, <code>authController.js</code>
express-session	1.19.0	Session HTTP	verrou, <code>server.js</code>
express-mysql-session	3.0.3	Persistance MySQL des sessions	verrou, <code>server.js</code>
cookie-parser	1.4.7	Lecture des cookies	verrou, <code>server.js</code>
csurf	1.11.0	Protection CSRF	verrou, <code>server.js</code>
express-rate-limit	8.5.2	Limitation globale de l'API et de la connexion	verrou, <code>server.js</code>
express-validator	7.3.2	Validation et normalisation des entrées	verrou, <code>validations/</code>
Helmet	8.2.0	En-têtes HTTP et CSP	verrou, <code>server.js</code>
Morgan	1.11.0	Journal HTTP	verrou, <code>server.js</code>
ExcelJS	4.4.0	Construction des classeurs XLSX	verrou, <code>exportController.js</code>
dotenv	17.4.2	Chargement de la configuration d'environnement	verrou, fichiers de configuration
nodemon	3.1.14	Dépendance de développement déclarée	verrou de dépendances
Bootstrap	5.3.3	Composants et mise en page	balises CDN des pages HTML

Technologie	Version exacte détectée	Rôle réel	Preuve
		frontend	
Font Awesome	6.5.2	Icônes	balises CDN des pages HTML
HTML/CSS/JavaScript	Standard navigateur	Pages, styles, interactions et appels API	<code>views/</code>

Les numéros ci-dessus proviennent du fichier de verrouillage, qui résout les plages déclarées dans `package.json`. Le moteur MySQL exact et sa version d'exécution ne sont pas déterminables sans connexion à une instance ; ils ne sont donc pas inventés.

3.3 Choix technologiques

Express réunit le service des pages et les endpoints JSON dans un même processus, ce qui correspond au caractère monolithique de l'application. MySQL offre les contraintes relationnelles, les transactions, les index et les verrous nécessaires aux sélections, aux plannings, aux congés et aux sessions. `mysql2/promise` permet d'écrire ces flux avec `async/await`.

L'interface sans framework JavaScript applicatif réduit la chaîne de construction : les fichiers HTML peuvent être servis directement et chaque page charge son script. Bootstrap fournit les composants et la grille, tandis que la feuille `leoni.css` porte les adaptations visuelles spécifiques. ExcelJS répond au besoin de générer un classeur structuré ; l'export CSV est construit par le contrôleur.

Les bibliothèques de sécurité se répartissent les responsabilités : `bcrypt` pour les secrets, `express-session` pour le cycle de session, le magasin MySQL pour la persistance, `Helmet` pour les en-têtes, `csrf` pour le jeton anti-CSRF, `express-rate-limit` pour le débit et `express-validator` pour les données reçues. Le code combine ces protections au lieu de les traiter comme substituables.

3.4 Organisation des fichiers

Répertoire ou fichier	Responsabilité observée
<code>server.js</code>	Composition Express, montage des routes, erreurs et nettoyage périodique
<code>config/</code>	Constantes, permissions, menu, pool et initialisation des données
<code>routes/</code>	Définition des pages et endpoints, middlewares d'accès et validations
<code>controllers/</code>	Adaptation HTTP, orchestration et certaines requêtes directes
<code>services/</code>	Règles métier, transactions, verrous et orchestration de modèles
<code>models/</code>	Requêtes de données pour les domaines structurés

Répertoire ou fichier	Responsabilité observée
middlewares/	Authentification, onboarding, permission et validation
validations/	Chaînes express-validator par module
utils/	Erreurs, journalisation, async handler, fonctions communes et fenêtre
views/	Pages HTML et erreurs 403/404
views/assets/	CSS, scripts navigateur, favicon
sql/schema.sql	Schéma consolidé des tables métier
sql/migrations/	Sept migrations numérotées
tests/	Trois fichiers de tests Node natifs

Les sources historiques externes au code, les secrets d’environnement, les dépendances installées, les métadonnées et les éléments de gestion de versions sont exclus du périmètre documentaire. La structure réellement inspectée correspond aux répertoires ci-dessus.

3.5 Configuration du serveur Express

Le serveur active `trust proxy` avec la valeur 1, utile lorsque l’application se trouve derrière un proxy unique. L’ordre des middlewares est significatif :

1. Helmet et la politique CSP ;
2. journalisation HTTP Morgan ;
3. limitation générale des appels `/api` et limitation de la connexion ;
4. lecture des cookies, des formulaires et du JSON ;
5. service des assets statiques ;
6. session Express persistée dans MySQL ;
7. protection CSRF ;
8. routes de pages ;
9. routes API d’authentification, utilisateurs, planning, sélection mensuelle, tableau de bord, export, audit, congés et sessions de travail ;
10. réponse 404 ;
11. gestionnaire global des erreurs.

Le cookie de session se nomme `leoni_session`, dure huit heures, est inaccessible au JavaScript grâce à `HttpOnly`, utilise `SameSite=Lax` et devient `Secure` lorsque l’environnement est la production. L’application n’est pas démarrée dans le cadre de ce rapport, de sorte que l’efficacité dynamique de ces réglages reste à valider sur un environnement d’essai.

3.6 Connexion et accès à MySQL

La configuration de base construit un pool `mysql2/promise` avec un maximum de dix connexions. Les paramètres proviennent de variables d'environnement dont seules les désignations génériques sont documentées dans le guide d'installation ; aucune valeur n'est reproduite. Le pool est partagé pour les lectures simples. Les services qui exigent une transaction demandent une connexion dédiée, appellent `beginTransaction`, puis `commit` ou `rollback` dans un bloc protégé.

Le code d'initialisation est exécuté lors du chargement de la configuration. Il peut créer ou faire évoluer le schéma et comporte une étape d'amorçage d'un compte initial. Cette automatisation facilite une première installation, mais couple le démarrage HTTP à des mutations de schéma et à une logique de bootstrap. Pour un environnement gouverné, une procédure de migration explicite, versionnée et exécutée séparément offrirait un meilleur contrôle. Aucun identifiant ou secret d'amorçage n'est reproduit dans cette documentation.

Le paramètre `multipleStatements` est activé pour l'initialisation. Les requêtes métier de premier niveau inspectées utilisent des paramètres pour les valeurs. L'activation reste néanmoins une capacité sensible : toute future requête non maîtrisée dans ce contexte devrait être revue avec attention.

3.7 Gestion des sessions

`express-session` conserve l'état d'authentification et délègue la persistance à `express-mysql-session`. La table technique correspondante est gérée à l'exécution par la bibliothèque ; elle est distincte des six tables métier du schéma consolidé. La session applicative contient un objet utilisateur réduit, comprenant les attributs nécessaires à l'identité, au rôle et à l'onboarding, ainsi que les permissions calculées lors de la lecture de session.

Le middleware `auth` refuse les requêtes sans utilisateur de session. `requireOnboardingComplete` bloque les pages et API métiers tant que le mot de passe doit être remplacé. `requirePermission` compare la permission demandée à la liste accordée au rôle. La déconnexion détruit la session et efface le cookie correspondant.

La durée maximale du cookie est de huit heures. Le code ne met pas en évidence une rotation périodique indépendante du cookie ni un écran de gestion des sessions multiples. Ces absences sont des constats de périmètre, non une preuve d'exploitation.

3.8 Authentification

La page `/login` contient un champ de type e-mail et un champ mot de passe. Le script envoie `{email, password}` à `POST /api/auth/login`. Le validateur normalise l'e-mail et contrôle les données. Le contrôleur recherche le compte actif par e-mail, compare le mot de passe avec `bcrypt`, place le profil réduit en session et inscrit l'action de connexion dans le journal.

`GET /api/auth/session` renseigne le frontend sur l'utilisateur courant, son onboarding et ses permissions. `POST /api/auth/logout` ferme la session. Un limiteur dédié réduit le nombre de tentatives sur l'endpoint de connexion.

Les messages de refus distinguent actuellement le compte absent du mot de passe incorrect. Cette précision facilite le diagnostic pour l'utilisateur mais peut révéler si une adresse est enregistrée. Une formulation uniforme est une perspective de durcissement. Aucun mécanisme de réinitialisation administrative du mot de passe par le Team Leader n'est exposé dans les routes actuelles ; le flux confirmé est le changement personnel avec l'ancien mot de passe.

[CAPTURE À INSÉRER : interface de connexion avec champs vides et aucune donnée personnelle]

Figure 16 — Authentification par e-mail.

[CAPTURE À INSÉRER : rendu du diagramme `05\_sequence\_authentification.puml`]

Figure 5 — Séquence d'authentification.

3.9 Changement obligatoire du mot de passe

Deux drapeaux de users, `first_login` et `must_change_password`, déclenchent le blocage d'onboarding. Lorsqu'ils sont actifs, la route de connexion ou les middlewares orientent l'utilisateur vers `/change-password?reason=password-required`. Le formulaire demande l'ancien mot de passe, le nouveau et sa confirmation. La validation impose une longueur minimale de huit caractères et la cohérence des données ; le contrôleur vérifie l'ancien secret, produit un nouveau hash `bcrypt`, désactive les drapeaux et journalise `PASSWORD_CHANGED`.

La route reste accessible à tout utilisateur authentifié afin de permettre aussi un changement volontaire. L'obligation repose simultanément sur les indicateurs persistés, la session et le middleware de pages/API.

[CAPTURE À INSÉRER : page de changement obligatoire avec tous les champs de mot de passe vides]

Figure 17 — Changement obligatoire du mot de passe.

**[CAPTURE À INSÉRER : rendu du diagramme
`06\_sequence\_changement\_mot\_de\_passe.puml`]**

Figure 6 — Séquence de renouvellement du mot de passe.

3.10 Gestion des rôles et permissions

Les constantes définissent deux rôles immuables et le fichier de permissions associe à chacun une liste d'actions atomiques. Le Team Leader reçoit vingt permissions couvrant les statistiques, les utilisateurs, les lectures et générations globales, les sélections mensuelles, les synthèses, les congés, les exports, l'audit et les paramètres. Data Cleansing reçoit six permissions : tableau de bord, lecture et génération de son planning, lecture et écriture de sa sélection, et lecture de ses congés.

`requirePermission` protège les routes déclarées. Le frontend reçoit les permissions depuis la session et filtre les éléments de `SIDEBAR_ITEMS`. Comme ce filtrage client n'est pas une barrière de sécurité, les pages sensibles répètent les contrôles côté serveur. Les services appliquent en plus des règles de propriété, notamment pour le planning, les congés et les sessions.

[CAPTURE À INSÉRER : page 403 obtenue avec un compte d'essai sans permission, sans URL sensible]

Figure 31 — Refus d'un accès non autorisé.

3.11 Gestion des utilisateurs

La page `/users-page` et les endpoints `/api/users` sont réservés au Team Leader. Le module liste les comptes actifs, ouvre une modale de création ou de modification et propose une suppression. Les validateurs vérifient l'identifiant de route, les champs textuels, l'e-mail, le rôle et le mot de passe requis à la création. Le contrôleur utilise des requêtes paramétrées, génère un hash bcrypt et renseigne les indicateurs d'onboarding pour le nouveau compte.

La suppression est logique : le compte demeure en base mais porte `is_deleted = 1` et une date `deleted_at`. Les lectures courantes excluent ces lignes. Ce choix conserve les relations et les traces d'audit. Les actions de création, modification et suppression sont journalisées.

L'interface expose des champs d'identité et d'organisation tels que nom, prénom, matricule, département, poste ou statut selon les colonnes et le formulaire. Les contraintes uniques sur le nom d'utilisateur, l'e-mail et le matricule complètent les contrôles applicatifs. Aucun mécanisme de pagination n'est visible : la liste entière correspondant à la requête est chargée.

[CAPTURE À INSÉRER : page de gestion des utilisateurs avec identités, e-mails et matricules masqués]

Figure 20 — Répertoire des utilisateurs.**[CAPTURE À INSÉRER : modale de création vide, sans mot de passe saisi]****Figure 21 — Formulaire de création d'un utilisateur.****[CAPTURE À INSÉRER : rendu du diagramme `07\_sequence\_gestion\_utilisateur.puml`]****Figure 7 — Séquence de création ou de mise à jour d'un utilisateur.****3.12 Sélection mensuelle du groupe Home Office**

La sélection mensuelle remplace une affectation fixe pour la génération. `monthly_group_selections` relie un utilisateur, une clé `YYYY-MM` et un groupe 1 ou 2. La contrainte unique utilisateur/mois rend le choix univoque. `GET /api/monthly-group-selections/mine` lit le choix personnel ; `PUT /mine` l'enregistre ; `GET /` fournit au Team Leader l'état d'un mois.

Le service utilise une transaction. Il obtient la date métier depuis MySQL, vérifie que la fenêtre est ouverte et que le mois reçu est exactement le mois suivant. Il recherche un planning existant et refuse la modification s'il y en a un. En l'absence de verrou, il insère ou met à jour la sélection, puis distingue dans l'audit une sélection initiale d'un changement.

La vue de planning peut afficher la sélection d'un compte cible au Team Leader, mais l'endpoint d'écriture reste volontairement limité au propre compte de l'utilisateur connecté. Le champ historique `users.group_id` subsiste pour d'autres usages ; le générateur courant exige la sélection mensuelle et n'en déduit pas automatiquement un groupe ancien.

[CAPTURE À INSÉRER : zone de sélection du groupe pour le mois autorisé, identité de tiers masquée]**Figure 22 — Sélection mensuelle du groupe Home Office.****[CAPTURE À INSÉRER : rendu du diagramme `08\_sequence\_selection\_groupe.puml`]****Figure 8 — Séquence de sélection du groupe.****3.13 Fenêtre de génération**

La fenêtre est un mécanisme partagé entre la sélection et la génération. Le code métier ne s'appuie pas uniquement sur la date du navigateur. Il interroge MySQL pour une heure UTC, la convertit dans `Africa/Tunis`, construit le mois suivant et calcule le dernier jour réel du mois courant. L'ouverture commence le 25 inclus et se prolonge jusqu'à ce dernier jour inclus.

Le service rejette les mois passés, le mois courant et les mois trop éloignés. Il traite naturellement les mois de 28, 29, 30 ou 31 jours et le passage de décembre à janvier. Une première lecture fournit l'état à l'interface ; les services d'écriture répètent la vérification dans

leur transaction, ce qui évite de se fier à une information devenue obsolète entre l’affichage et la soumission.

Le tableau de bord calcule également des états visuels dans le navigateur. Ceux-ci sont informatifs ; la décision d’autoriser l’écriture appartient au backend. Une divergence d’horloge entre navigateur et serveur pourrait donc modifier l’affichage sans contourner la garde serveur.

3.14 Algorithme de génération du planning

Le service commence par identifier l’utilisateur cible et vérifier qu’un utilisateur ordinaire ne génère pas pour autrui. Dans une transaction, il contrôle la fenêtre, verrouille les données utiles, charge la sélection exacte du mois et refuse tout planning déjà présent. Il parcourt ensuite chaque jour calendaire du mois cible.

Pour le **groupe A**, un jour est retenu s’il s’agit d’un mercredi, d’un jeudi ou d’un vendredi dont le rang dans le mois est impair : premier, troisième et, lorsqu’il existe, cinquième. Pour le **groupe B**, les lundis et mardis sont retenus, ainsi que les deuxième et quatrième vendredis. Le rang d’un vendredi est obtenu à partir du jour du mois. Le lot final est inséré avec le statut `remote`, `work_hour = 0` et `planned_work_hour = 8`.

Pseudo-code fidèle au comportement :

```

vérifier acteur, cible, fenêtre et mois suivant
ouvrir une transaction
revalider la fenêtre avec la date MySQL en fuseau Tunis
charger et verrouiller la sélection utilisateur/mois
refuser si aucune sélection ou si un planning existe
pour chaque date du mois cible :
    si groupe A et (mercredi ou jeudi ou vendredi impair) : retenir
    si groupe B et (lundi ou mardi ou vendredi pair) : retenir
insérer en lot les dates retenues comme journées distantes
journaliser la génération
valider la transaction ; sinon l’annuler

```

La table autorise aussi le statut `onsite`, mais l’algorithme courant ne crée que les jours distants calculés ; il ne matérialise pas toutes les autres dates comme `onsite`. Le champ `horaire` n’est pas renseigné par ce flux.

[CAPTURE À INSÉRER : zone de génération affichant fenêtre, mois cible et groupe, avec identité masquée]

Figure 23 — Génération du planning mensuel.

**[CAPTURE À INSÉRER : rendu du diagramme
`09\_sequence\_generation\_planning.puml`]**

Figure 9 — Séquence transactionnelle de génération.

3.15 Consultation du planning

Les endpoints de lecture prennent en charge la consultation filtrée, la vue globale, la vue d'un utilisateur et les données du calendrier. Les paramètres de mois, groupe ou utilisateur sont contrôlés avant d'atteindre les requêtes. Le service établit le périmètre : un utilisateur sans `planning.read.all` reste limité à son propre identifiant, même si une route de lecture ne déclare pas explicitement une permission fine.

Les lignes retournent les dates distantes et les colonnes utiles, notamment le statut, les heures réalisées, les heures prévues et le champ `horaire`. La page de planning fournit des filtres, une cible lorsque le rôle le permet, la sélection mensuelle, le bouton de génération et la table des résultats. Le champ `horaire` est affiché comme une valeur absente lorsqu'il est nul ; aucun calcul ou flux d'écriture courant n'a été trouvé pour le renseigner.

La contrainte unique `user_id/date` empêche deux lignes pour la même personne et la même journée. L'unicité mensuelle de génération est renforcée par la vérification du service avant l'insertion.

[CAPTURE À INSÉRER : tableau de planning filtré avec identités et données internes masquées]

Figure 24 — Consultation du planning Home Office.

**[CAPTURE À INSÉRER : rendu du diagramme
`10\_sequence\_consultation\_planning.puml`]**

Figure 10 — Séquence de consultation du planning.

3.16 Calendrier

La page `/calendar-page` exige la permission de lecture personnelle du planning. Son script demande les calendriers à l'API, regroupe les résultats par utilisateur et par mois, puis construit une représentation mensuelle. Le Team Leader peut recevoir un périmètre plus large selon les permissions et filtres appliqués par le backend ; Data Cleansing reste centré sur ses données.

Le calendrier est une projection des lignes persistées, non un second moteur de génération. Il ne crée ni ne modifie les dates. La cohérence dépend donc des mêmes données que la table de planning. L'interface doit être validée avec des mois courts, longs et commençant à différents jours de la semaine, car aucun test navigateur automatisé n'est présent.

[CAPTURE À INSÉRER : calendrier mensuel avec identité masquée]

Figure 26 — Visualisation calendaire des jours distants.

3.17 Tableau de bord

GET `/api/dashboard/stats` est protégé par l'authentification et la fin de l'onboarding. Le contrôleur effectue plusieurs agrégations sur les utilisateurs et le planning afin de fournir des volumes, des groupes, des statuts ou des indicateurs de validation utilisés par l'interface. La page construit des cartes, des taux et, pour le Team Leader, un tableau de suivi.

Le navigateur déduit aussi le prochain mois à partir de son horloge et affiche des bannières de statut ou des toasts. Ces messages sont des éléments d'interface éphémères : aucune table ni service de notifications persistantes, aucun envoi d'e-mail et aucune intégration de messagerie n'apparaissent dans le code. La décision d'écriture du planning reste contrôlée par la date serveur.

Une réserve d'autorisation est observable : la route de statistiques n'exige pas `dashboard.statistics`, alors que cette permission n'est accordée qu'au Team Leader. Toute personne authentifiée ayant terminé l'onboarding atteint le contrôleur. Selon les agrégats renvoyés, cela rend des statistiques globales accessibles au profil Data Cleansing. Cette portée devrait être validée et, si nécessaire, alignée sur la permission déclarée.

[CAPTURE À INSÉRER : tableau de bord Team Leader avec chiffres, noms et matricules internes masqués]

Figure 18 — Tableau de bord du Team Leader.

[CAPTURE À INSÉRER : tableau de bord Data Cleansing avec données personnelles masquées]

Figure 19 — Tableau de bord Data Cleansing.

3.18 Export CSV et XLSX

Les routes `/api/export/csv` et `/api/export/xlsx` exigent respectivement `export.csv` et `export.xlsx`, deux permissions du Team Leader. Le contrôleur lit les filtres de groupe, d'utilisateur et de mois, construit des conditions SQL à partir de fragments contrôlés et passe les valeurs comme paramètres. Les colonnes exportées portent sur l'identifiant, l'utilisateur, le matricule, le nom, la date distante et les heures.

Pour le CSV, le serveur construit le contenu textuel et définit les en-têtes de téléchargement. Pour XLSX, ExcelJS crée un classeur, une feuille, les colonnes et les lignes, puis écrit le flux HTTP. Les deux opérations produisent une action d'audit indiquant le type d'export et les filtres utiles sans que le rapport reproduise de données personnelles.

L'export porte sur l'ensemble du résultat filtré ; aucun mécanisme de pagination ou de génération en tâche de fond n'est visible. Sur un volume important, la mémoire et le temps de réponse devraient donc être mesurés.

[CAPTURE À INSÉRER : page d'export avec filtres, sans identité réelle]

Figure 29 — Configuration d'un export CSV ou XLSX.

[CAPTURE À INSÉRER : rendu du diagramme `14\_sequence\_export.puml`]

Figure 14 — Séquence d'export.

3.19 Journal d'audit

`audit_logs` contient l'utilisateur éventuel, le nom de l'action, un détail textuel, l'adresse IP et la date de création. `utils/appLogger.js` centralise l'insertion. Les constantes couvrent notamment la connexion, la déconnexion, le mot de passe, les comptes, la génération, les congés, les exports, les sessions de travail et les sélections mensuelles.

La route de lecture est réservée à `audit.read`. Le contrôleur joint les informations utilisateur et renvoie les cent événements les plus récents, limite fixée par `AUDIT_LOG_LIMIT`. La page affiche la date, l'acteur, l'action et le détail ; l'adresse IP enregistrée n'est pas présentée dans le tableau actuel.

Le journal constitue une trace applicative et ne doit pas être assimilé à une preuve inviolable. Le schéma ne montre ni signature, ni archivage immuable, ni politique de rétention configurable. Les détails peuvent contenir des identifiants métier ; les captures et exports documentaires doivent les masquer.

[CAPTURE À INSÉRER : page d'audit avec utilisateurs et détails sensibles masqués]

Figure 30 — Consultation des événements d'audit.

[CAPTURE À INSÉRER : rendu du diagramme `15\_sequence\_audit.puml`]

Figure 15 — Séquence de lecture du journal.

3.20 Gestion des congés

Le module reconnaît les types `annual`, `sick`, `exceptional`, `unpaid` et `other`, et les statuts `pending`, `approved`, `rejected`, `cancelled`. Chaque demande contient une date de début, une date de fin, un motif, puis éventuellement un commentaire de décision, un réviseur et une date de décision.

L'utilisateur consulte ses demandes par `GET /mine`, crée par `POST /` et annule une demande encore en attente par `PATCH /:id/cancel`. Les validations contrôlent le type, le format et l'ordre des dates ainsi qu'une limite de 500 caractères pour les textes concernés. Dans une

transaction, le service refuse le chevauchement avec une demande en attente ou approuvée du même utilisateur.

Le Team Leader lit toutes les demandes puis appelle les endpoints d'approbation ou de rejet. Le service verrouille la ligne, vérifie pending et refuse une décision sur sa propre demande. Il enregistre le réviseur, le commentaire et la décision, puis écrit l'audit. Une demande approuvée ne modifie pas automatiquement le planning et n'interrompt pas une session de travail ; les modules restent séparés dans le code actuel.

[CAPTURE À INSÉRER : formulaire et historique personnel de congés, motifs et dates sensibles masqués]

Figure 27 — Création et suivi d'une demande de congé.

[CAPTURE À INSÉRER : file de traitement du Team Leader, données personnelles masquées]

Figure 28 — Traitement des demandes de congé.

[CAPTURE À INSÉRER : rendu du diagramme `11\_sequence\_demande\_conge.puml`]

Figure 11 — Séquence de création d'une demande de congé.

[CAPTURE À INSÉRER : rendu du diagramme `12\_sequence\_traitement\_conge.puml`]

Figure 12 — Séquence d'approbation ou de rejet.

3.21 Suivi du travail à distance

Le suivi est lié à une ligne de planning `remote` appartenant au compte connecté et correspondant à la date du serveur. L'endpoint `auto-start` cherche d'abord une session éligible. Il peut reprendre une session en pause ou créer une nouvelle session active. La contrainte unique construite autour de `active_slot` protège l'existence d'une seule session active pour le triplet utilisateur, planning et date ; le service traite aussi un conflit concurrent en rechargeant la session existante.

Pendant l'activité, le navigateur appelle `heartbeat` toutes les 60 secondes. Il surveille seulement l'activité de la page pour déterminer une inactivité de cinq minutes et demander une pause ; le code n'enregistre ni écran, ni frappe, ni webcam, ni contenu d'activité. L'interface précise le caractère non intrusif du compteur. Le serveur utilise une grâce de 120 secondes pour évaluer le heartbeat.

Une pause, une fin ou une expiration cumule la durée depuis le dernier point actif dans `active_seconds`. Le service agrège ensuite les secondes de la journée, les convertit en heures et met à jour `planning.work_hour` sans dépasser 28 800 secondes, soit huit heures.

`planned_work_hour` demeure indépendant et vaut huit lors de la génération. Les statuts distinguent `active`, `paused`, `ended` et `expired`.

Un nettoyage des sessions obsolètes est lancé au démarrage du serveur puis toutes les deux minutes. Les opérations significatives — démarrage, reprise, pause, fin, expiration et mise à jour des heures — sont auditable. L’endpoint de synthèse est réservé au Team Leader. Son filtre de groupe s’appuie encore sur `users.group_id`, alors que la génération actuelle repose sur `monthly_group_selections`; cette différence peut produire un périmètre de synthèse qui ne reflète pas un choix mensuel récent.

[CAPTURE À INSÉRER : compteur et état d’une session distante de test, sans identité]

Figure 25 — État visible d’une session de travail.

[CAPTURE À INSÉRER : rendu du diagramme `13\_sequence\_session\_travail.puml`]

Figure 13 — Cycle d’une session de travail distant.

3.22 Gestion des validations

Les validateurs sont regroupés par domaine : authentification, utilisateurs, planning, sélection mensuelle, congés et sessions. Ils contrôlent notamment les e-mails, longueurs, identifiants entiers, rôles, groupes, clés de mois, dates ISO, types et statuts implicites. `middlewares/validate.js` rassemble les erreurs et empêche l’entrée dans le contrôleur en cas d’échec.

Ces contrôles applicatifs complètent les contraintes SQL. Par exemple, le groupe est contrôlé dans l’API et par la table ; l’unicité est vérifiée métier puis imposée par un index ; les statuts sont limités par des énumérations. Cette redondance fournit des messages contrôlés tout en conservant une barrière au niveau des données.

Les formulaires appliquent également des attributs HTML et des contrôles JavaScript pour l’expérience utilisateur. Ils ne remplacent pas la validation serveur, puisque les endpoints peuvent être appelés sans l’interface.

3.23 Gestion des erreurs

`utils/errors.js` définit des erreurs applicatives portant un statut HTTP et un caractère opérationnel. `asyncHandler` enveloppe plusieurs contrôleurs pour transmettre les rejets à Express. Le gestionnaire global de `server.js` renvoie les erreurs opérationnelles de manière concise et masque les détails ou traces internes pour une erreur inattendue.

Les services annulent leurs transactions dans un bloc `catch`, puis libèrent la connexion dans `finally`. Les contrôleurs traduisent aussi certains résultats en réponses 400, 401, 403, 404 ou

409. Le frontend commun intercepte les réponses non réussies, gère la perte de session et présente des messages ou toasts.

Le style n'est pas entièrement uniforme : certains contrôleurs envoient directement des statuts tandis que d'autres lèvent des classes d'erreur. Cette hétérogénéité n'empêche pas le fonctionnement, mais elle augmente le nombre de conventions qu'un mainteneur doit connaître.

3.24 Sécurité

Les protections observées sont complémentaires :

- hashage bcrypt avec dix tours et vérification de l'ancien mot de passe ;
- session serveur persistée dans MySQL et cookie `HttpOnly/SameSite=Lax` ;
- activation conditionnelle de `Secure` en production ;
- protection CSRF placée après la session ;
- Helmet et CSP limitant les origines de scripts, styles, polices et connexions ;
- limiteur général de l'API et limiteur de connexion ;
- validations serveur et contraintes SQL ;
- requêtes paramétrées pour les valeurs observées ;
- permissions de rôle, onboarding et contrôles de propriété ;
- suppression logique des comptes ;
- audit d'actions majeures ;
- masquage des erreurs internes inattendues.

Les limites de sécurité directement visibles sont également documentées : messages de connexion différenciés, amorçage couplé au démarrage, absence de flux administratif de réinitialisation, dépendance à une configuration correcte du secret de session et du mode production, absence de test automatisé des contrôles HTTP, et accès aux statistiques du tableau de bord sans permission `dashboard.statistics` explicite. Le rapport ne qualifie donc pas l'application de « parfaitement sécurisée » ; il décrit une défense en profondeur partielle dont le comportement doit être testé en environnement contrôlé.

3.25 Présentation des interfaces

Interface	Rôle autorisé	Composants et actions	Données et validations	Capture
Connexion	Visiteur	E-mail, mot de passe, soumission	E-mail valide, secret masqué, limitation serveur	Figure 16
Changement du mot de	Authentifié	Ancien secret, nouveau,	Longueur minimale et	Figure 17

Interface	Rôle autorisé	Composants et actions	Données et validations	Capture
passer		confirmation	ancien secret correct	
Tableau de bord	Tous après onboarding	Cartes, taux, bannière, tableau selon rôle	Agrégats API ; états calculés par le client	Figures 18–19
Utilisateurs	Team Leader	Tableau, recherche, modales, suppression	Identités, rôle, unicités et validation serveur	Figures 20–21
Planning et sélection	Tous selon périmètre	Fenêtre, groupe, génération, filtres, lignes	Mois, cible, propriété et règles métier	Figures 22–24
Session de travail	Propriétaire éligible	État, compteur, fin, message de confidentialité	Date serveur et planning distant personnel	Figure 25
Calendrier	Tous selon périmètre	Navigabilité mensuelle et cellules	Projection des jours persistés	Figure 26
Congés	Tous pour soi, TL pour gestion	Formulaire, historique, décisions	Dates, type, chevauchement, statut	Figures 27–28
Export	Team Leader	Filtres et boutons CSV/XLSX	Filtres appliqués côté serveur	Figure 29
Audit	Team Leader	Tableau des derniers événements	Limite de 100, données sensibles à masquer	Figure 30
Accès interdit	Utilisateur sans permission	Message 403	Aucun changement de données	Figure 31

Toutes les captures sont à réaliser sur une instance d’essai autorisée. Les identités, e-mails, matricules, motifs, commentaires et détails d’audit doivent être recouverts par des masques opaques. Si l’état nécessaire n’existe pas sans modifier la base, le placeholder est conservé.

3.26 Difficultés techniques observables et solutions implémentées

La première difficulté observable est la **frontière temporelle mensuelle**. Un simple contrôle du numéro de jour dans le navigateur serait fragile face au fuseau, aux fins de mois et aux requêtes concurrentes. La solution isole le calcul, utilise une date MySQL convertie dans Africa/Tunis, calcule le dernier jour réel et répète la garde dans les transactions. Les tests couvrent février, l’année bissextile et le passage d’année.

La deuxième difficulté est la **cohérence entre sélection et génération**. La table impose une sélection unique par mois ; le service verrouille, refuse une modification après création du planning et exige la sélection exacte lors de la génération. Le planning est inséré en lot et protégé par une unicité utilisateur/date.

La troisième difficulté est la **concurrence sur les sessions actives**. Le service cherche une session existante, expire les états obsolètes, utilise une contrainte d’unicité autour de `active_slot` et récupère l’enregistrement concurrent en cas de collision. Les transitions cumulent les secondes avant de modifier le statut.

La quatrième difficulté est la **séparation du prévu et du réalisé**. Les migrations distinguent `planned_work_hour` de `work_hour`; le générateur initialise respectivement huit et zéro, puis le service de session agrège le temps réel et applique le plafond journalier.

La cinquième difficulté est le **contrôle d'accès multi-rôle**. Les permissions atomiques, les middlewares, le menu filtré et les contrôles de propriété apportent plusieurs niveaux. La cohérence gagnerait encore à être renforcée par l'emploi explicite de permissions sur toutes les routes de lecture.

3.27 Limites actuelles du système

Les limites suivantes proviennent uniquement du code actuel :

1. **Couverture automatisée restreinte.** Trois fichiers testent surtout la fenêtre, la sélection et la génération ; aucun test automatisé dédié n'a été trouvé pour l'authentification, les utilisateurs, les congés, les sessions, l'export, l'audit, les interfaces ou une base MySQL réelle.
2. **Script de test non raccordé.** `npm test` est encore un placeholder qui termine en erreur, alors que `node --test tests/*.test.js` exécute la suite existante.
3. **Version Node non contrainte.** `package.json` ne déclare pas `engines`, ce qui laisse la compatibilité d'exécution à documenter.
4. **Architecture de données hétérogène.** Certains domaines utilisent service et modèle, d'autres interrogent le pool depuis le contrôleur.
5. **Mutations au démarrage.** Le chargement de la base peut créer ou modifier le schéma et amorcer un compte ; les migrations ne sont pas séparées du démarrage applicatif.
6. **Champ `horaire` peu exploité.** La colonne existe et l'interface affiche une valeur absente, mais aucun calcul ou enregistrement métier courant n'a été identifié.
7. **Statistiques globales insuffisamment cloisonnées.** `/api/dashboard/stats` n'emploie pas la permission `dashboard.statistics` pourtant déclarée.
8. **Permissions de lecture du planning peu explicites.** Plusieurs routes dépendent du filtrage du contrôleur/service au lieu d'annoncer la permission dans le routeur.
9. **Référence de groupe duale.** La génération utilise le choix mensuel, tandis que la synthèse des sessions filtre encore sur `users.group_id`.
10. **Notifications éphémères.** Les bannières et toasts sont calculés dans le navigateur ; aucun centre persistant ou canal externe n'est présent.
11. **Horloge client pour certains affichages.** Le tableau de bord déduit le mois cible depuis le navigateur, alors que le backend utilise une date MySQL en fuseau Tunis.

12. **Congés non intégrés au planning.** Une approbation ne modifie pas automatiquement les lignes du planning ou les sessions.
13. **Absence de pagination visible.** Les utilisateurs, plannings, demandes et journaux sont chargés selon des limites fixes ou l'ensemble filtré, sans navigation paginée généralisée.
14. **Export synchrone.** CSV et XLSX sont produits dans la requête HTTP, sans file de tâches ni traitement par lots.
15. **Dépendance aux CDN.** Bootstrap, les icônes et la police exigent un accès aux ressources distantes pour un rendu complet.
16. **Libellés linguistiques mixtes.** L'interface combine des termes français et anglais, ce qui réduit l'homogénéité éditoriale.
17. **Aucune documentation API intégrée.** Aucun schéma OpenAPI ou portail d'API n'est visible.
18. **Aucun pipeline de déploiement ou supervision visible.** La source ne montre ni intégration continue, ni conteneurisation, ni métriques applicatives dédiées.
19. **Flux de mot de passe limité.** Le projet expose le changement personnel, sans endpoint dédié de réinitialisation administrative ou de récupération.
20. **Messages de connexion différenciés.** Ils peuvent révéler si un e-mail existe.

Ces éléments n'annulent pas les fonctions confirmées. Ils définissent le périmètre actuel et orientent des travaux futurs vérifiables.

3.28 Conclusion

La réalisation associe un serveur Express, un schéma relationnel, des règles transactionnelles et un frontend directement servi. Les modules centraux sont opérationnellement reliés par l'utilisateur, le mois et le planning. Le traitement de la fenêtre et des sessions montre une attention particulière aux dates et à la concurrence. La sécurité repose sur plusieurs mécanismes, avec des points de consolidation clairement identifiables. Le prochain chapitre évalue les preuves d'exécution disponibles et formalise les essais manuels restant nécessaires.

Chapitre 4 — Tests et validation

4.1 Introduction

La validation distingue trois niveaux de preuve. Le premier est l'exécution réelle de la suite automatisée. Le deuxième est la vérification syntaxique de tous les fichiers JavaScript applicatifs, qui ne valide que leur analyse par Node.js. Le troisième est l'inspection statique du code, utile pour confirmer l'existence d'un mécanisme mais insuffisante pour conclure à son fonctionnement complet en situation. Les parcours nécessitant un serveur, un navigateur ou une base d'essai restent explicitement « À valider ».

4.2 Stratégie de validation

La commande prescrite a été exécutée depuis `leoni-planing/` sans démarrer le serveur et sans se connecter à une base réelle. Les tests natifs remplacent les dépendances de données lorsque nécessaire. Une commande `node --check` a ensuite analysé chaque fichier JavaScript de premier niveau applicatif, hors dépendances.

Pour les modules non automatisés, un catalogue de tests manuels est préparé. Il spécifie les préconditions, les étapes et le résultat attendu, mais conserve « Non exécuté » tant qu'aucun essai contrôlé n'a été réalisé. Cette séparation évite de confondre une route lisible avec un parcours fonctionnel validé.

4.3 Tests automatisés existants

Trois fichiers sont présents :

1. `planningGenerationWindow.test.js` teste la construction et la validation de la fenêtre temporelle ;
2. `monthlyGroupSelectionWindowGuard.test.js` teste les gardes entourant l'enregistrement de la sélection mensuelle ;
3. `planningServiceGenerationGuard.test.js` teste les refus du service et la génération des groupes A et B avec des doubles de données.

Les scénarios couvrent la fermeture avant le 25, l'ouverture à partir du 25, les mois de longueur différente, février, l'année bissextile, le passage décembre-janvier, le fuseau Tunis, le mois suivant, l'absence de sélection, le planning existant et les deux groupes.

4.4 Résultats réels

Élément	Résultat observé
Date	15 juillet 2026
Commande	<code>node --test tests/*.test.js</code>
Node.js de l'environnement	v24.16.0
npm détecté	11.13.0
Tests	19
Succès	19
Échecs	0
Annulés	0
Ignorés	0
TODO	0
Durée rapportée	116,323958 ms
Code de sortie	0

La vérification syntaxique complémentaire a également réussi pour tous les fichiers JavaScript applicatifs contrôlés. Elle ne démontre ni l'accès à MySQL, ni le rendu des pages, ni l'efficacité des mécanismes de sécurité.

4.5 Validation de l'authentification

ID	Module	Préconditions	Étapes	Résultat attendu	Résultat observé	Statut
TM-01	Connexion valide	Compte d'essai actif	Saisir e-mail et mot de passe valides	Session créée et redirection correcte	Non exécuté	À valider
TM-02	Mot de passe erroné	Compte d'essai actif	Soumettre un secret incorrect	Refus 401, aucune session	Non exécuté	À valider
TM-03	Compte supprimé	Compte d'essai désactivé	Tenter la connexion	Accès refusé	Non exécuté	À valider
TM-04	Limitation	Environnement d'essai isolé	Répéter des tentatives contrôlées	Réponse de limitation après seuil	Non exécuté	À valider
TM-05	Déconnexion	Session active	Utiliser la déconnexion puis rappeler une page privée	Session détruite et redirection connexion	Non exécuté	À valider

L'existence de ces contrôles est confirmée par inspection, mais aucun test d'intégration d'authentification n'est présent dans la suite exécutée.

4.6 Validation des autorisations

ID	Module	Préconditions	Étapes	Résultat attendu	Résultat observé	Statut
TM-06	Page utilisateurs	Session Data Cleansing	Ouvrir <code>/users-page</code>	Réponse 403	Non exécuté	À valider

ID	Module	Préconditions	Étapes	Résultat attendu	Résultat observé	Statut
TM-07	API utilisateurs	Session Data Cleansing et jeton CSRF	Appeler POST <code>/api/users</code>	Réponse 403, aucune écriture	Non exécuté	À valider
TM-08	Export	Session Data Cleansing	Appeler un endpoint d'export	Réponse 403	Non exécuté	À valider
TM-09	Planning d'autrui	Session Data Cleansing	Demander l'identifiant d'un autre compte	Données d'autrui non renvoyées	Non exécuté	À valider
TM-10	Tableau de bord	Deux rôles de test	Comparer les statistiques reçues	Périmètre conforme à la décision métier	Non exécuté	À valider

Le dernier test est particulièrement important, car la permission `dashboard.statistics` est déclarée mais absente de la route `/stats`.

4.7 Validation des utilisateurs

ID	Module	Préconditions	Étapes	Résultat attendu	Résultat observé	Statut
TM-11	Création	Team Leader et données d'essai uniques	Soumettre le formulaire	Compte créé, mot de passe hashé, onboarding actif, audit	Non exécuté	À valider
TM-12	Unicité	Donnée d'essai déjà utilisée	Recréer le même e-mail ou matricule	Refus contrôlé, aucune duplication	Non exécuté	À valider
TM-13	Mise à jour	Compte d'essai existant	Modifier un champ autorisé	Donnée persistée et audit	Non exécuté	À valider
TM-14	Suppression logique	Compte d'essai sans session utile	Supprimer puis relire	<code>is_deleted</code> actif, compte exclu des listes et de la connexion	Non exécuté	À valider

4.8 Validation de la sélection mensuelle

ID	Scénario automatisé	Résultat observé	Statut
TA-09	Enregistrement hors fenêtre	Écriture refusée et transaction annulée	Réussi
TA-10	Mois différent du mois suivant	Requête refusée	Réussi
TA-11	Sélection A/B valide	Enregistrement simulé, commit et audit confirmés	Réussi

La suite couvre les gardes du service avec des substituts. Un essai MySQL doit encore confirmer l'index unique, le verrou et l'upsert dans l'environnement cible.

4.9 Validation de la génération du planning

ID	Scénario automatisé	Résultat observé	Statut
TA-01	Fenêtre fermée du 1er au 24	Comportement attendu confirmé	Réussi

ID	Scénario automatisé	Résultat observé	Statut
TA-02	Fenêtre ouverte du 25 à la fin du mois	Bornes inclusives confirmées	Réussi
TA-03	Mois de 30 et 31 jours	Fin réelle utilisée	Réussi
TA-04	Février non bissextile	Fin au 28 confirmée	Réussi
TA-05	Février bissextile	Fin au 29 confirmée	Réussi
TA-06	Passage décembre-janvier	Mois et année suivants confirmés	Réussi
TA-07	Fuseau Africa/Tunis	Date métier confirmée	Réussi
TA-08	Mois cible	Seul le mois suivant accepté	Réussi
TA-12	Génération hors fenêtre	Aucun lot inséré	Réussi
TA-13	Sélection d'un ancien mois	Sélection non réutilisée	Réussi
TA-14	Sélection absente	Génération refusée	Réussi
TA-15	Planning existant	Doublon refusé	Réussi
TA-16	Groupe A	Dates générées conformes dans le mois	Réussi
TA-17	Groupe B	Dates générées conformes dans le mois	Réussi

Les tests paramétrés expliquent que le test runner compte 19 tests alors que le tableau consolide 17 familles de scénarios sur les deux modules. Le total du runner est la mesure officielle.

4.10 Validation du calendrier

ID	Module	Préconditions	Étapes	Résultat attendu	Résultat observé	Statut
TM-15	Mois complet	Planning d'essai existant	Ouvrir le calendrier et naviguer	Jours placés aux dates correctes	Non exécuté	À valider
TM-16	Périmètre	Sessions TL et Data Cleansing	Comparer les utilisateurs visibles	Chacun voit uniquement le périmètre autorisé	Non exécuté	À valider
TM-17	Responsive	Navigateur aux trois largeurs de rupture	Redimensionner et naviguer	Aucun chevauchement ou contenu inaccessible	Non exécuté	À valider

4.11 Validation des congés

ID	Module	Préconditions	Étapes	Résultat attendu	Résultat observé	Statut
TM-18	Création	Utilisateur et dates d'essai libres	Créer une demande valide	Statut pending et audit	Non exécuté	À valider
TM-19	Chevauchement	Demande pending/approved existante	Soumettre une plage recouvrante	Refus contrôlé	Non exécuté	À valider
TM-20	Annulation	Demande personnelle pending	Annuler	Statut cancelled	Non exécuté	À valider

ID	Module	Préconditions	Étapes	Résultat attendu	Résultat observé	Statut
TM-21	Approbation	TL et demande d'autrui pending	Approuver	Statut, réviseur et date renseignés	Non exécuté	À valider
TM-22	Auto-traitement	Demande du TL connecté	Approuver ou rejeter	Refus	Non exécuté	À valider
TM-23	Décision répétée	Demande déjà traitée	Tenter une seconde décision	Refus sans modification	Non exécuté	À valider

4.12 Validation des sessions de travail

ID	Module	Préconditions	Étapes	Résultat attendu	Résultat observé	Statut
TM-24	Démarrage	Planning distant personnel à la date serveur	Appeler auto-start	Session active créée ou reprise	Non exécuté	À valider
TM-25	Date non éligible	Aucun planning distant du jour	Appeler auto-start	Refus, aucune session	Non exécuté	À valider
TM-26	Heartbeat	Session active	Laisser le navigateur actif	Heartbeat actualisé toutes les 60 s	Non exécuté	À valider
TM-27	Pause d'inactivité	Session active et aucune activité pendant cinq minutes	Attendre dans l'environnement d'essai	Session mise en pause	Non exécuté	À valider
TM-28	Propriété	Session appartenant à un autre compte	Tenter pause ou fin	Refus	Non exécuté	À valider
TM-29	Expiration	Session active avec heartbeat obsolète	Exécuter le nettoyage	Statut expired , temps cumulé	Non exécuté	À valider
TM-30	Plafond	Plusieurs périodes totalisant plus de huit heures	Terminer les périodes	work_hour ne dépasse pas 8	Non exécuté	À valider
TM-31	Concurrence	Deux demandes de démarrage simultanées	Appeler en parallèle	Une seule session active	Non exécuté	À valider

4.13 Validation des exports

ID	Module	Préconditions	Étapes	Résultat attendu	Résultat observé	Statut
TM-32	CSV	TL et données d'essai	Exporter avec filtres	Fichier lisible, colonnes et lignes conformes, audit	Non exécuté	À valider
TM-33	XLSX	TL et données d'essai	Exporter avec filtres	Classeur ouvrable, types et en-têtes conformes, audit	Non exécuté	À valider
TM-34	Filtre	Plusieurs mois/groupes/utilisateurs d'essai	Comparer les exports	Uniquement les lignes filtrées	Non exécuté	À valider
TM-35	Accès interdit	Data Cleansing	Appeler les endpoints	Réponse 403	Non exécuté	À valider

4.14 Validation de l'audit

ID	Module	Préconditions	Étapes	Résultat attendu	Résultat observé	Statut
TM-36	Écriture	Réaliser chaque action majeure avec des comptes d'essai	Lire la table ou la page	Action, acteur, détail et date cohérents	Non exécuté	À valider
TM-37	Limite	Plus de cent événements d'essai autorisés	Ouvrir la page	Les cent plus récents seulement	Non exécuté	À valider
TM-38	Autorisation	Data Cleansing	Ouvrir / logs - page et /api/logs	Réponse 403	Non exécuté	À valider
TM-39	Données affichées	Team Leader	Comparer table et page	Adresse IP non affichée ; détails correctement rendus	Non exécuté	À valider

4.15 Validation de la base de données

ID	Module	Préconditions	Étapes	Résultat attendu	Résultat observé	Statut
TM-40	Installation	Instance MySQL d'essai vide et sauvegardable	Exécuter la procédure autorisée	Six tables métier et table technique de session disponibles	Non exécuté	À valider
TM-41	Contraintes	Données d'essai	Tenter les doublons interdits	Rejet par les index uniques	Non exécuté	À valider
TM-42	Relations	Données d'essai liées	Supprimer/désactiver selon le flux prévu	Effets FK conformes et comptes logiquement conservés	Non exécuté	À valider
TM-43	Transactions	Provoquer une erreur contrôlée en environnement jetable	Vérifier l'état final	Aucune écriture partielle	Non exécuté	À valider
TM-44	Index	Volume représentatif anonymisé	Examiner les plans des requêtes critiques	Index utilisés de manière adaptée	Non exécuté	À valider

Aucune connexion MySQL n'a été ouverte pour cette mission documentaire. Le schéma et les migrations ont été inspectés statiquement.

4.16 Validation des interfaces

ID	Module	Préconditions	Étapes	Résultat attendu	Résultat observé	Statut
TM-45	Navigation	Comptes d'essai des deux rôles	Parcourir le menu	Éléments conformes aux permissions	Non exécuté	À valider
TM-46	CSRF frontend	Session active	Soumettre chaque formulaire	Jeton inclus et écriture acceptée/refusée correctement	Non exécuté	À valider
TM-47	Erreurs	Simuler des réponses 400/401/403/409/500 sans donnée	Observer l'interface	Message compréhensible, aucune donnée interne révélée	Non exécuté	À valider

ID	Module	Préconditions	Étapes	Résultat attendu	Résultat observé	Statut
		réelle				
TM-48	Responsive	Largeurs bureau, tablette et mobile	Parcourir toutes les pages	Navigation et actions utilisables	Non exécuté	À valider
TM-49	Clavier	Navigateur moderne	Naviguer sans souris	Focus visible et commandes accessibles	Non exécuté	À valider
TM-50	Mouvement réduit	Préférence système activée	Charger et utiliser l'application	Animations réduites conformément au CSS	Non exécuté	À valider

4.17 Couverture et limites des tests

La couverture exécutée est forte sur un noyau précis : calendrier de la fenêtre, mois cible, sélection et gardes de génération. Elle vérifie des cas limites importants sans base réelle. Elle ne mesure toutefois ni un pourcentage de lignes, ni les branches de l'ensemble du projet. Aucun outil de couverture n'est configuré.

Les modules d'identité, de contrôle d'accès, de données administratives, de congé, de session, d'export et d'audit reposent uniquement sur l'inspection dans cette mission. Les contrôles CSRF, CSP, cookies et limiteurs exigent un serveur. Les contraintes et transactions exigent une instance MySQL isolée. Le rendu et l'accessibilité exigent des essais navigateur. Ces validations ne doivent pas être déclarées réussies tant que les lignes TM-01 à TM-50 restent non exécutées.

4.18 Bilan des résultats

Le bilan vérifié est double : **19 tests automatisés sur 19 ont réussi**, et **tous les fichiers JavaScript applicatifs contrôlés passent l'analyse syntaxique de Node.js**. Ce résultat confirme le périmètre explicitement testé et l'absence d'erreur de syntaxe détectée dans les sources JavaScript.

Il ne permet pas d'affirmer que l'ensemble des parcours est validé en exécution. Les 50 scénarios manuels proposés constituent un plan de recette reproductible à mener sur une base d'essai autorisée, avec des comptes et données non sensibles.

4.19 Conclusion

La suite existante apporte une preuve fiable pour les règles temporelles et de génération. La stratégie de validation recommandée prolonge cette base par des tests d'intégration et d'interface, sans confondre leur préparation avec leur résultat. Cette transparence rend le bilan exploitable : les acquis sont mesurés et les zones non exécutées sont identifiées sans ambiguïté.

Conclusion générale

Le projet documenté répond à une problématique de centralisation du planning Home Office et de plusieurs opérations associées. L'application réunit dans une même interface l'identité des utilisateurs, la sélection mensuelle d'un groupe, la génération contrôlée du prochain mois, la consultation tabulaire et calendaire, les congés, les sessions de travail à distance, les exports et l'audit. Deux rôles structurent les accès : le Team Leader dispose d'un périmètre d'administration et de supervision, tandis que Data Cleansing travaille principalement sur ses propres données.

La réalisation repose sur un monolithe Node.js/Express connecté à MySQL. Les couches routes, contrôleurs, services et modèles sont particulièrement visibles dans les domaines transactionnels, même si plusieurs contrôleurs accèdent directement à la base. Le frontend reste volontairement direct, avec des pages HTML, une feuille CSS partagée, Bootstrap et des scripts JavaScript natifs. Le client API commun relie ces pages aux endpoints JSON.

Le mécanisme de planning constitue le noyau technique. Une horloge de référence issue de MySQL et convertie dans le fuseau `Africa/Tunis` ouvre la préparation du 25 au dernier jour réel du mois. Le système n'accepte que le mois suivant, exige une sélection mensuelle et refuse la duplication. Le groupe A produit les mercredis, jeudis et vendredis impairs ; le groupe B produit les lundis, mardis et vendredis pairs. Transactions, verrous et contraintes uniques protègent ce flux.

Le suivi du travail distant complète le planning sans mécanisme intrusif. Il démarre ou reprend une session liée à une journée distante personnelle, reçoit un heartbeat, gère pause, fin et expiration, cumule les secondes actives puis plafonne les heures réalisées à huit. Les heures prévues et réalisées demeurent séparées. Le module de congés apporte son propre cycle de statuts et empêche le chevauchement actif ainsi que l'auto-traitement d'une demande par le Team Leader.

La sécurité observable combine bcrypt, sessions MySQL, attributs de cookie, CSRF, Helmet et CSP, limitation de requêtes, validation des entrées, requêtes paramétrées, permissions et audit. Elle doit être comprise comme un ensemble de contrôles implémentés, non comme une garantie absolue. Les réserves identifiées — autorisation des statistiques, messages d'authentification différenciés, bootstrap au démarrage et couverture de tests partielle — fournissent des axes précis de consolidation.

La suite automatisée a été réellement exécutée : les 19 tests ont tous réussi. Elle confirme la fenêtre temporelle, le mois cible, la sélection et les gardes de génération. Le reste du système a

été inspecté mais n'a pas été exécuté dans cette mission documentaire. Le rapport propose donc cinquante tests manuels sans leur attribuer un résultat fictif.

Les compétences et contributions personnelles doivent enfin être exprimées par l'étudiant :

[INFORMATION À COMPLÉTER : compétences techniques et professionnelles effectivement acquises, responsabilités assumées et retour d'expérience]. L'état de déploiement doit également être confirmé : **[INFORMATION À COMPLÉTER : environnement réel et niveau de validation de l'application]**.

En synthèse, le code révèle une application fonctionnellement riche, structurée autour de règles mensuelles explicites et de contrôles multi-couches. La documentation produite fournit une base académique et technique vérifiable, ainsi qu'un plan clair pour terminer la validation et préparer la soutenance.

Perspectives

Les perspectives ci-dessous sont des travaux futurs dérivés des limites visibles. Elles ne sont pas présentées comme des fonctions existantes.

P-01 — Élargir la couverture automatisée

Ajouter des tests unitaires et d'intégration pour l'authentification, les autorisations, les utilisateurs, les congés, les sessions de travail, les exports et l'audit. Une base MySQL éphémère permettrait de vérifier réellement les contraintes, verrous et annulations. Des tests navigateur couvriraient le menu, les formulaires, les erreurs et les largeurs responsives.

P-02 — Raccorder la commande de test standard

Remplacer le placeholder du script `npm test` par la commande de la suite et introduire un rapport de couverture. Cette évolution réduirait le risque qu'un opérateur conclue à tort que les tests n'existent pas.

P-03 — Uniformiser l'architecture des données

Déplacer progressivement les requêtes directes des contrôleurs vers des services et des modèles cohérents. Les contrôleurs resteraient centrés sur HTTP, les services sur les règles et les modèles sur SQL. Cette homogénéisation faciliterait l'injection de doubles et les tests.

P-04 — Séparer migrations et démarrage

Créer une commande de migration explicite, réexécutable et contrôlée, puis retirer les mutations de schéma et l'amorçage du chemin de démarrage ordinaire. Les secrets ou identifiants initiaux devraient provenir d'un mécanisme d'administration sécurisé, jamais d'une valeur codée.

P-05 — Aligner toutes les autorisations

Appliquer `dashboard.statistics` aux statistiques destinées au Team Leader et déclarer explicitement les permissions de lecture sur chaque route de planning. Des tests de matrice par rôle devraient empêcher une régression du périmètre.

P-06 — Unifier la notion de groupe

Faire reposer les synthèses mensuelles sur `monthly_group_selections` pour le mois demandé, ou documenter clairement l'usage distinct du champ historique `users.group_id`. Cette évolution éviterait une différence entre le groupe utilisé pour générer et celui utilisé pour filtrer une synthèse.

P-07 — Consolider les fonctions opérationnelles

Décider du rôle métier du champ `horaire`, relier éventuellement une absence approuvée au planning selon une règle validée, ajouter une pagination aux listes et traiter les exports volumineux de façon progressive ou asynchrone. Chacune de ces évolutions exige une décision métier préalable.

P-08 — Améliorer les notifications et la cohérence temporelle

Si le besoin est confirmé, introduire un centre de notifications persistant et calculer les états mensuels depuis une date renvoyée par le serveur. Cela éviterait de dépendre de l'horloge du navigateur et permettrait de retrouver un message après rechargement.

P-09 — Renforcer l'exploitation

Définir une version Node supportée, une procédure de configuration, un pipeline de validation, un déploiement reproductible, des sauvegardes, des journaux structurés, des métriques et des alertes. Ces fonctions ne sont pas visibles actuellement et leur choix dépend de l'infrastructure autorisée.

P-10 — Documenter l'API et homogénéiser l'interface

Produire un contrat OpenAPI à partir des routes et validations, puis harmoniser les libellés français/anglais. Une vérification d'accessibilité et la possibilité d'héberger localement les assets CDN consolideraient l'usage sur le réseau interne.

Annexes

Annexe A — Structure du projet

leoni-planing/	
├─ config/	constantes, permissions, menu et base
├─ controllers/	adaptation HTTP et orchestration
├─ middlewares/	authentification et validation
├─ models/	accès aux données par domaine
├─ routes/	pages et API
├─ services/	règles métier et transactions
├─ sql/	
│ └─ schema.sql	schéma consolidé
│ └─ migrations/	migrations 001 à 007
├─ tests/	trois fichiers de tests natifs Node
├─ utils/	erreurs, logs et fonctions partagées
├─ validations/	chaînes express-validator
├─ views/	
│ └─ assets/css/	feuille de style
│ └─ assets/js/	client API et scripts de pages
├─ package.json	
├─ package-lock.json	
└─ server.js	

Les secrets d'environnement, dépendances installées, métadonnées et documents externes au code sont volontairement absents de cet inventaire.

Annexe B — Catalogue des routes de pages

Méthode	Chemin	Accès	Résultat
GET	/	Public	Redirection vers <code>/login</code>
GET	<code>/login</code>	Public, avec redirection si session	Page de connexion
GET	<code>/change-password</code>	Authentifié	Page de changement du mot de passe
GET	<code>/dashboard</code>	Authentifié, onboarding terminé	Tableau de bord
GET	<code>/users-page</code>	Permission <code>users.read</code>	Gestion des utilisateurs
GET	<code>/planning-page</code>	Authentifié, onboarding terminé	Planning et sélection
GET	<code>/calendar-page</code>	Permission <code>planning.read.own</code>	Calendrier
GET	<code>/leave-requests-page</code>	Permission <code>leave_requests.read.own</code>	Congés
GET	<code>/export-page</code>	Permission <code>export.csv</code>	Export
GET	<code>/logs-page</code>	Permission <code>audit.read</code>	Audit

Annexe C — Catalogue des routes API

Groupe	Méthode et chemin	Protection ou finalité
Authentification	GET /api/auth/session	État de session
Authentification	POST /api/auth/login	Validation et limitation de connexion
Authentification	POST /api/auth/logout	Destruction de session
Authentification	POST /api/auth/change-password	Authentifié, validation
Utilisateurs	GET /api/users	users.read
Utilisateurs	POST /api/users	users.create, validation
Utilisateurs	PUT /api/users/:id	users.update, validation
Utilisateurs	DELETE /api/users/:id	users.delete, suppression logique
Fenêtre	GET /api/planning/generation-window	planning.generate.own
Planning	POST /api/planning/generate	planning.generate.own, cible contrôlée
Planning	GET /api/planning/calendar	Authentifié, périmètre service
Planning	GET /api/planning	Authentifié, filtres et périmètre service
Planning	GET /api/planning/all	Authentifié, périmètre service
Planning	GET /api/planning/:user_id	Authentifié, périmètre service
Sélection	GET /api/monthly-group-selections/mine	Lecture personnelle
Sélection	PUT /api/monthly-group-selections/mine	Écriture personnelle et transaction
Sélection	GET /api/monthly-group-selections	Lecture globale TL
Tableau de bord	GET /api/dashboard/stats	Authentifié, onboarding terminé
Congés	GET /api/leave-requests/mine	Lecture personnelle
Congés	POST /api/leave-requests	Création personnelle validée
Congés	PATCH /api/leave-requests/:id/cancel	Annulation personnelle
Congés	GET /api/leave-requests	leave_requests.manage
Congés	PATCH /api/leave-requests/:id/approve	Gestion TL, validation
Congés	PATCH /api/leave-requests/:id/reject	Gestion TL, validation
Sessions	POST /api/work-sessions/auto-start	Planning personnel éligible
Sessions	POST /api/work-sessions/heartbeat	Session personnelle
Sessions	POST /api/work-sessions/pause	Session personnelle
Sessions	POST /api/work-sessions/end	Session personnelle
Sessions	GET /api/work-sessions/mine	Historique personnel filtré

Groupe	Méthode et chemin	Protection ou finalité
Sessions	GET /api/work-sessions/summary	Synthèse Team Leader
Export	GET /api/export/csv	export.csv
Export	GET /api/export/xlsx	export.xlsx
Audit	GET /api/logs	audit.read

Toutes les API d'écriture sont placées sous la protection CSRF globale après la session. Les chemins sont reconstitués depuis les points de montage de `server.js` et les routeurs relatifs.

Annexe D — Matrice complète des permissions

Permission	Team Leader	Data Cleansing
dashboard.read	Oui	Oui
dashboard.statistics	Oui	Non
users.read	Oui	Non
users.create	Oui	Non
users.update	Oui	Non
users.delete	Oui	Non
planning.read.own	Oui	Oui
planning.read.all	Oui	Non
planning.generate.own	Oui	Oui
planning.generate.all	Oui	Non
monthly_group_selection.read.own	Oui	Oui
monthly_group_selection.write.own	Oui	Oui
monthly_group_selection.read.all	Oui	Non
work_sessions.read.summary	Oui	Non
leave_requests.read.own	Oui	Oui
leave_requests.manage	Oui	Non
export.csv	Oui	Non
export.xlsx	Oui	Non
audit.read	Oui	Non
settings.manage	Oui	Non

`settings.manage` est déclaré dans la matrice mais aucun module de paramètres correspondant n'est monté dans les routes actuelles. Sa présence représente une capacité configurée mais non matérialisée par une interface ou un endpoint identifié.

Annexe E — Résumé des tables

Table	PK	FK principales	Unicités et index	Cycle de vie
users	id	—	username/e-mail/ matricule uniques ; index rôle/groupe/suppression	Timestamps et suppression logique
monthly_group_selections	id	utilisateur, cascade	utilisateur/mois unique ; index mois/groupe	Création/mise à jour, verrou métier après planning
planning	id	utilisateur, cascade	utilisateur/date unique ; index mois	Créé en lot, heures mises à jour
work_sessions	id	utilisateur cascade, planning mise à null	index utilisateur/date/statut ; unicité active	active → paused/ended/expired
leave_requests	id	demandeur cascade, réviseur mise à null	index utilisateur/statut/dates	pending → approved/rejected/cancelled
audit_logs	id	utilisateur mise à null	index date/action	Ajout uniquement par les flux observés

Annexe F — Catalogue synthétique des règles

Les règles RG-01 à RG-36 sont détaillées à la section 2.7. Elles se répartissent comme suit :

- RG-01 à RG-05 : authentification et mot de passe ;
- RG-06 à RG-09 : rôles, administration et unicités des utilisateurs ;
- RG-10 à RG-22 : date métier, sélection, génération et consultation du planning ;
- RG-23 à RG-26 : congés ;
- RG-27 à RG-34 : sessions de travail et heures ;
- RG-35 à RG-36 : export et audit.

La liste exhaustive est également reprise dans `01_audit_projet_actuel.md` et tracée par `04_matrice_tracabilite.md`.

Annexe G — Catalogue des tests

Fichier	Domaine	Nature
planningGenerationWindow.test.js	Fenêtre, fuseau, fins de mois et mois cible	Tests unitaires avec <code>node:test</code>
monthlyGroupSelectionWindowGuard.test.js	Garde de sélection mensuelle	Service avec substituts et transaction simulée
planningServiceGenerationGuard.test.js	Refus et dates des groupes A/B	Service avec substituts et lot simulé

Résultat global du 15 juillet 2026 : **19 réussis, 0 échoué, 0 ignoré**. Les cinquante scénarios TM préparés dans le chapitre 4 restent à valider tant qu'ils ne sont pas exécutés dans un environnement d'essai.

Annexe H — Liste des captures

1. connexion ;
2. changement obligatoire du mot de passe ;
3. tableau de bord Team Leader ;
4. tableau de bord Data Cleansing ;
5. gestion des utilisateurs ;
6. création d'un utilisateur ;
7. sélection mensuelle ;
8. génération ;
9. consultation du planning ;
10. session de travail ;
11. calendrier ;
12. congés personnels ;
13. traitement des congés ;
14. export ;
15. audit ;
16. accès interdit.

Le cadrage détaillé, la navigation, les données à masquer et les légendes figurent dans `03_plan_captures.md`. Aucune capture fictive n'est incluse.

Annexe I — Liste des diagrammes

Quinze sources PlantUML sont préparées dans `rapport_stage/diagrams/` : cas d'utilisation, architecture, composants, données, authentification, mot de passe, utilisateur, sélection, génération, consultation, création et traitement de congé, session de travail, export et audit. Le fichier `diagrams/README.md` donne les commandes de rendu. Les images ne sont pas intégrées tant qu'un moteur PlantUML local n'est pas disponible et que leur rendu n'a pas été vérifié.

Annexe J — Guide d’installation sans secrets

Ce guide décrit uniquement les étapes génériques déduites du projet. Il ne confirme pas un déploiement de production.

1. installer une version de Node.js compatible à valider, ainsi qu’une instance MySQL d’essai ;
2. copier le projet dans un répertoire autorisé ;
3. exécuter `npm ci` depuis `leoni-planing/` pour reproduire le verrou de dépendances ;
4. créer une configuration locale à partir de l’exemple, sans publier ses valeurs ;
5. renseigner les variables génériques nécessaires : `PORT`, `NODE_ENV`, `DB_HOST`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`, `SESSION_SECRET`, `RATE_LIMIT_WINDOW_MS`, `RATE_LIMIT_MAX`, `LOGIN_RATE_LIMIT_MAX`, `LOG_LEVEL` ;
6. préparer une base vide ou de recette conformément aux règles de l’organisation ;
7. examiner le comportement d’initialisation de `config/db.js` avant tout premier démarrage, car il peut modifier le schéma et amorcer un compte ;
8. démarrer dans l’environnement autorisé avec `npm start` ;
9. exécuter séparément `node --test tests/*.test.js` ;
10. vérifier les pages, les permissions et les journaux avec des comptes d’essai.

Ne jamais intégrer les valeurs de configuration au rapport, au code client, aux captures ou aux journaux partagés. Pour la production, une procédure spécifique de migration, de sauvegarde, de rotation des secrets et de supervision doit être définie par les responsables techniques.

Annexe K — Glossaire

- **Groupe A / B** : choix mensuel qui détermine les jours distants générés.
- **Heures prévues** : valeur attendue enregistrée séparément des heures réellement calculées.
- **Heures réalisées** : résultat du cumul des secondes de sessions, plafonné à huit heures par journée.
- **Heartbeat** : signal périodique d’une interface active vers le serveur.
- **Onboarding** : étape imposant le remplacement initial du mot de passe.
- **Planning distant** : ligne `remote` créée pour une date calculée.
- **Permission** : action atomique accordée à un rôle.
- **Session HTTP** : état d’authentification conservé côté serveur et référencé par un cookie.
- **Session de travail** : enregistrement métier des périodes actives d’une journée distante.
- **Suppression logique** : conservation de la ligne utilisateur avec indicateur de désactivation.

- **Transaction** : ensemble d'opérations SQL validé ou annulé comme une unité.

Le glossaire complet figure dans `08_glossaire.md`.

Références

Sources internes

1. Projet `leoni-planing` : code source actuel de `server.js`, `config/`, `routes/`, `controllers/`, `services/`, `models/`, `middlewares/`, `validations/`, `utils/` et `views/`.
2. Manifeste et verrou des dépendances : `package.json` et `package-lock.json`.
3. Schéma et migrations : `sql/schema.sql` et `sql/migrations/001` à `007`.
4. Tests : les trois fichiers du répertoire `tests/`.
5. Exemple de configuration non sensible : `.env.example`, limité aux noms de variables.

Documentations officielles consultées le 15 juillet 2026

1. Express.js, *5.x API Reference* : <https://expressjs.com/en/5x/api/>.
2. Node.js, *Test runner* : <https://nodejs.org/api/test.html>.
3. MySQL, *Documentation* : <https://dev.mysql.com/doc/>.
4. MySQL2, *Documentation* : <https://sidorares.github.io/node-mysql2/docs>.
5. express-session, *Documentation officielle* : <https://github.com/expressjs/session>.
6. express-validator, *Documentation* : <https://express-validator.github.io/docs/>.
7. Helmet.js, *Documentation officielle* : <https://helmetjs.github.io/>.
8. Bootstrap 5.3, *Introduction* : <https://getbootstrap.com/docs/5.3/getting-started/introduction/>.
9. ExcelJS, *Dépôt officiel* : <https://github.com/exceljs/exceljs>.
10. bcrypt pour Node.js, *Dépôt officiel* : <https://github.com/kelektiv/node.bcrypt.js>.
11. csurf, *Dépôt officiel* : <https://github.com/expressjs/csurf>.
12. PlantUML, *Documentation officielle* : <https://plantuml.com/>.

Les documentations externes expliquent les technologies générales. Les règles, rôles, routes, tables et résultats propres au projet restent fondés sur les sources internes.